

COMSATS University Islamabad, Lahore Campus



Lab Manual 1

“Introduction to MS SQL Server , DML and DDL”

Database Systems

Spring 2026

Department of Computer Science
CUI, Lahore, Pakistan

Table of Contents

1. Objective	2
2. Task Distribution	2
3. Getting Started with Microsoft SQL server	3
Step 1: Open SQL server management Studio.....	3
Step 2: Connect to Server	3
Step 3: Create your own Database	4
>>If you want to delete your database write the following query and Execute, make sure that the command is executed successfully.	5
You can also create new database by Left click on Database from object Explorer and select new Database, give it a name and press ok (as shown in following figures).	5
Step 4: View and use you database.....	7
4. Data Definition Language:.....	7
Step 1: Create table:	8
Step 2: Altering table Schema.....	8
Adding new column to existing table	8
Drop existing Column from existing table.....	8
Step 3: Add Foreign key Constraint.....	9
One way to create FK is to add it whilst creating table using the following syntax	9
Other way is to add FK constraint after creating the table, by using ALTER query,	9
Step 4: Add foreign key constraint to tables	10
Add FK whilst creating table.	10
FK can also be added after the referencing and referred tables have been created.....	12
5. Data Modification Language	12
Step 1: Insert rows into table	12
Step 2: To see the data from you table use the following table	14
Step 3: Delete rows from the table.....	14
Step 4: Delete all the data from the table.....	14
Exercise.....	16
Appendix.....	16
Comments is SQL	16
Data Types	Error! Bookmark not defined.
Where Conditions	16

1. Objective

The purpose of this lab is to introduce students to Microsoft SQL server environment and getting started with Data definition queries and Data modification queries.

2. Prerequisites

You should know the concepts of Primary key, foreign key and how to identify these constraints from the given schema. You have to read the manual before coming to lab.

3. Task Distribution

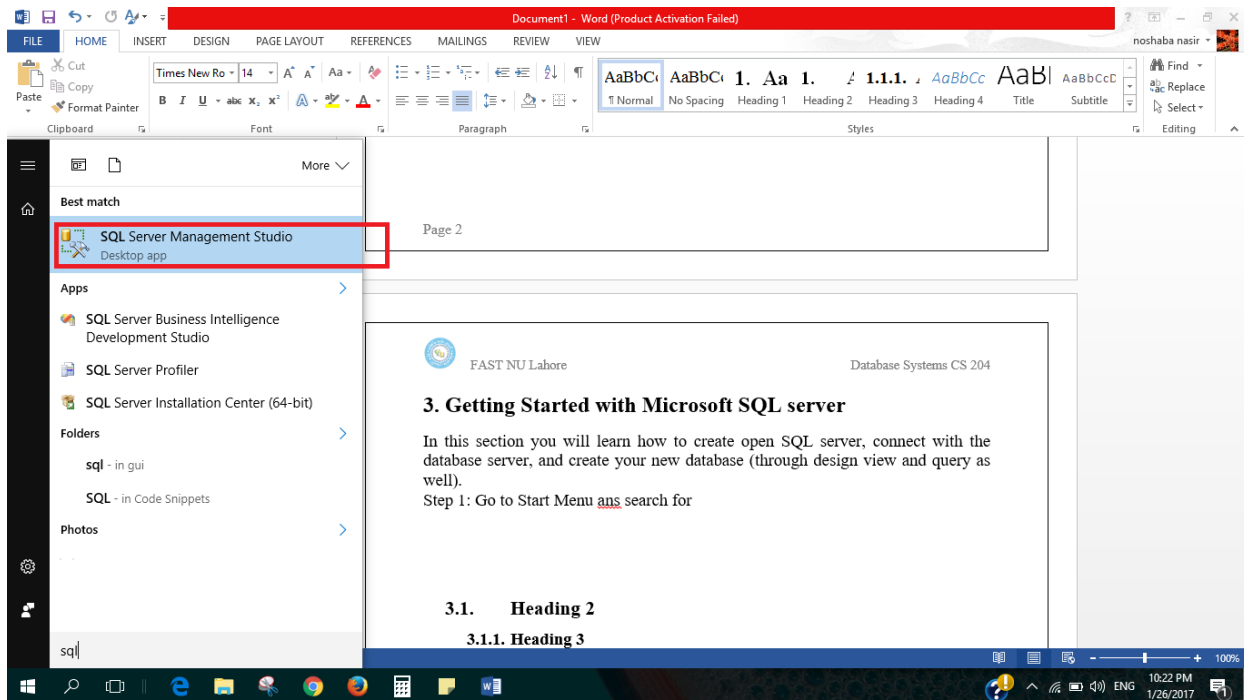
Total Time	180 Minutes
Introduction to MSSQL	20 Minutes
DDL	30 Minutes
DML	30 Minutes
Exercise	60 Minutes
Evaluation	40 Min

4. Getting Started with Microsoft SQL server

In this section you will learn how to create open SQL server, connect with the database server, and create your new database (through design view and query as well).

Step 1: Open SQL server management Studio

Go to Start Menu and search for SQL Server Management Studio, open it



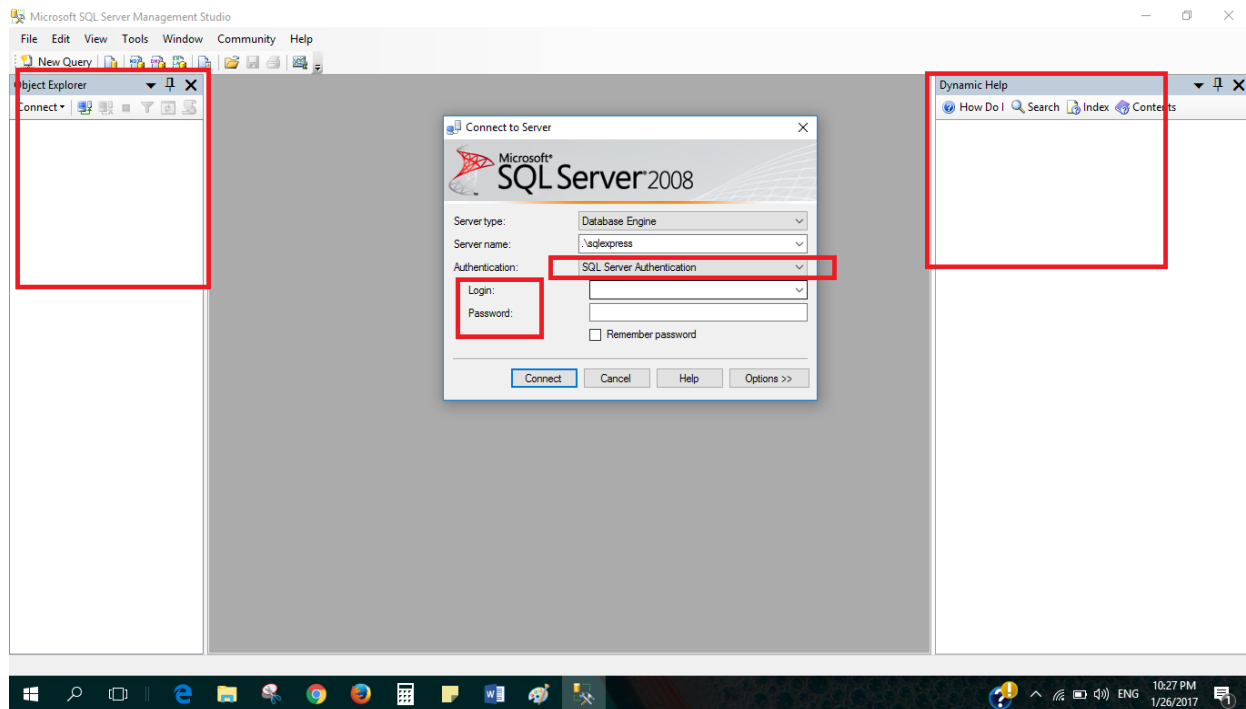
Step 2: Connect to Server

On the left side you have Object Explorer which you will use to explore all your databases and any object you create such as tables, on the right there is Dynamic Help.

In the middle you can see the Connect to server dialogue box.

Select SQL Server Authentication from the Authentication Drop down.

Ask for login and password from your lab instructor and press Connect (refer to following figure)

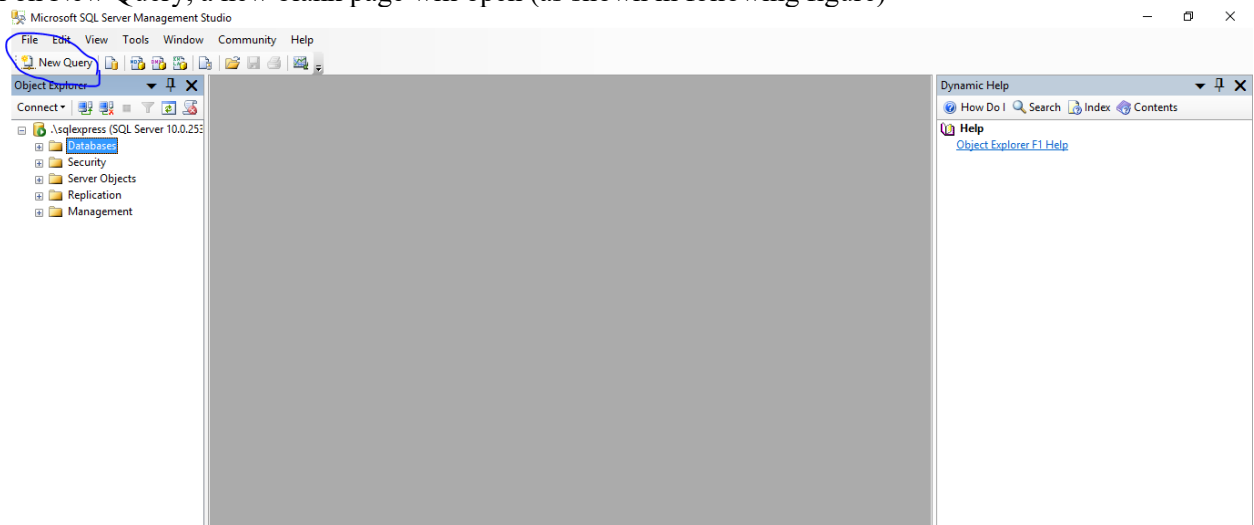


Step 3: Create your own Database

There are two ways to create a new data base

- Through Query

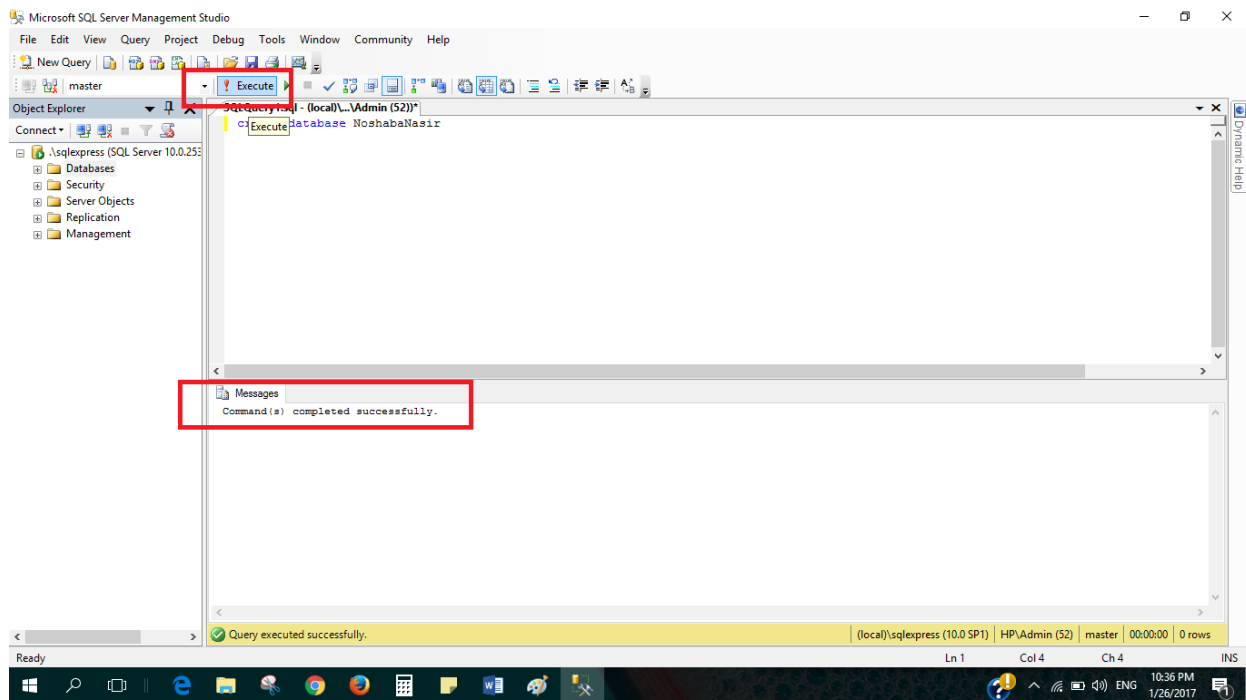
>>Click on New Query, a new blank page will open (as shown in following figure)



>>Write the following query on this page

```
createdatabase<nameofyouDatabase>
```

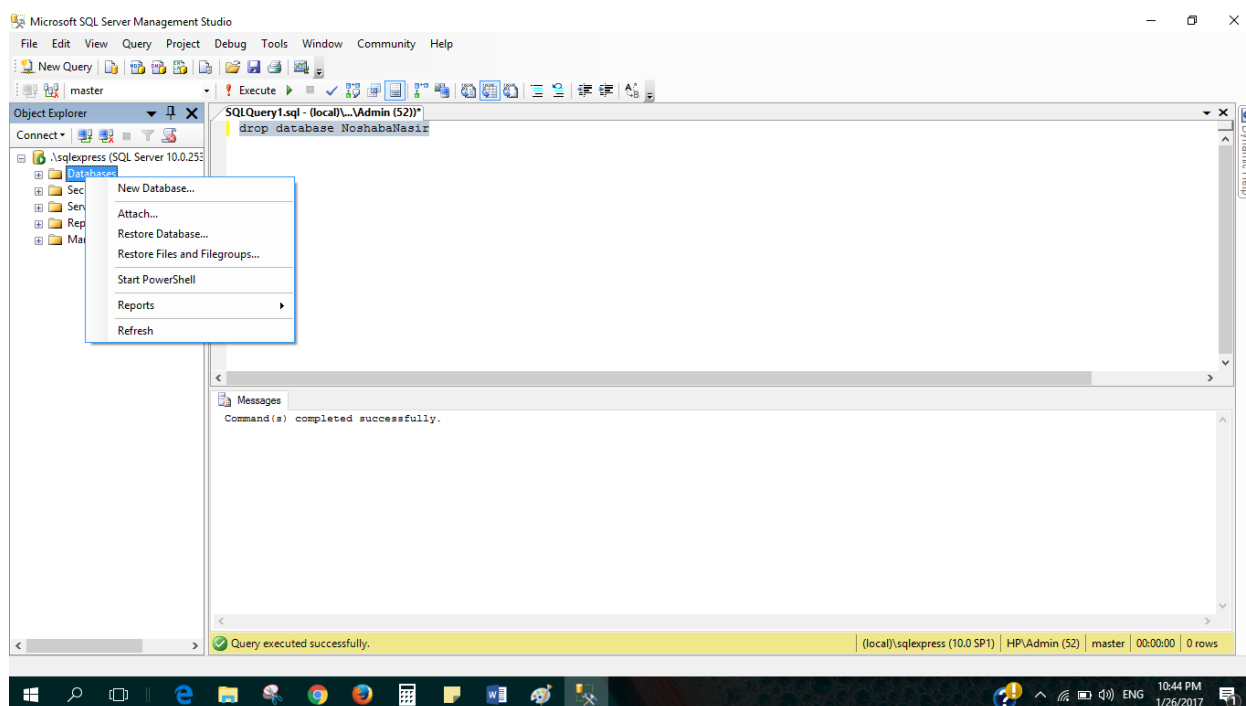
>>Hit Execute button shown on top, Command completed successfully shows that you database has been created. (As shown in following figure)

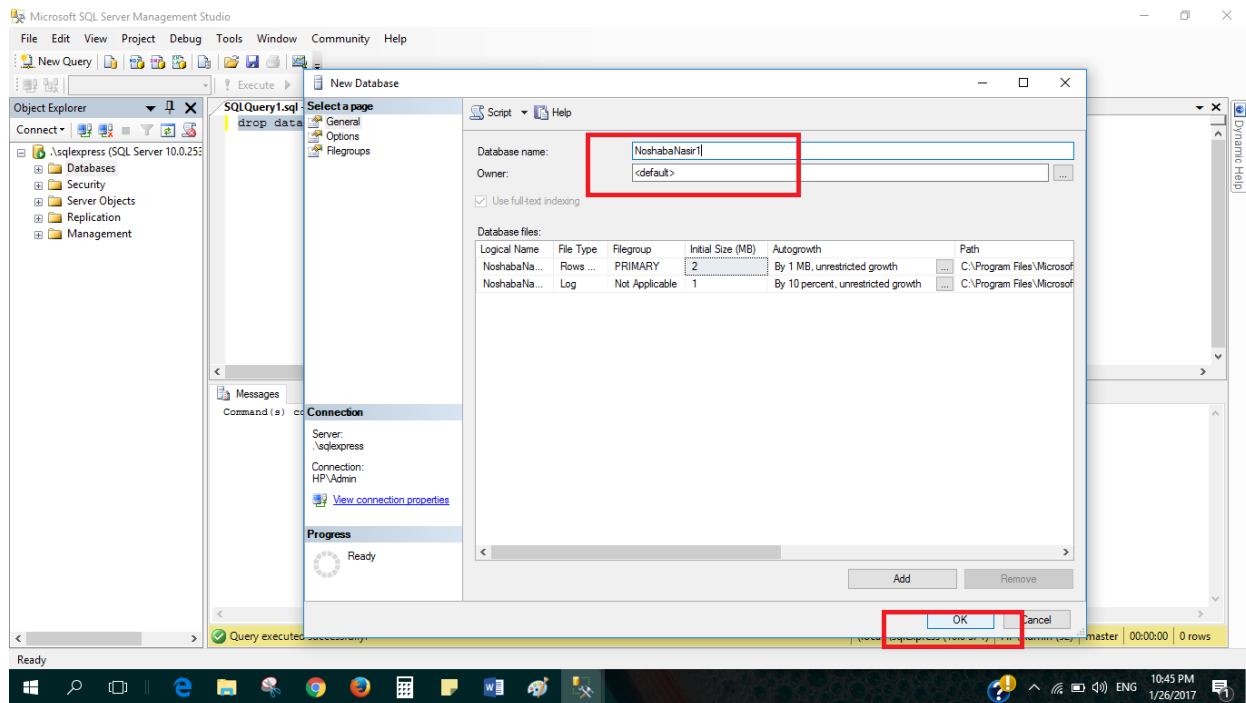


>>If you want to delete your database write the following query and Execute, make sure that the command is executed successfully.

`drop database <DatabaseName>`

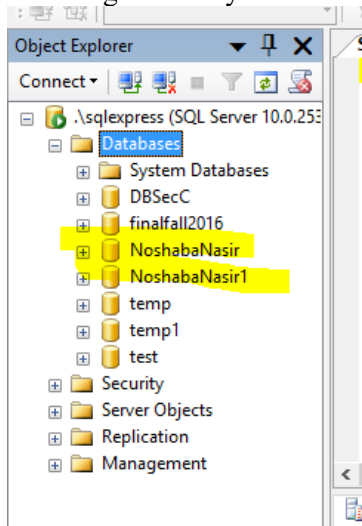
You can also create new database by Left click on Database from object Explorer and select new Database, give it a name and press ok (as shown in following figures).



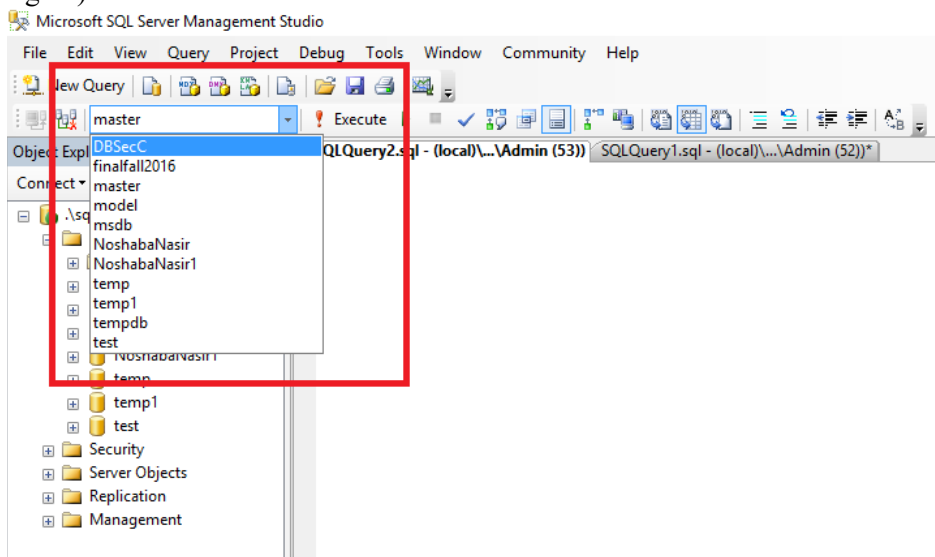


Step 4: View and use you database

If you expand the Database from object explorer, you can see all of the databases that exist of the server, including the newly created ones (as shown in following figure).



To use your database, make sure to select your database from drop down menu (as shown in following figure).



Or you can use the following query to use your database.

Use <databaseName>

5. Data Definition Language:

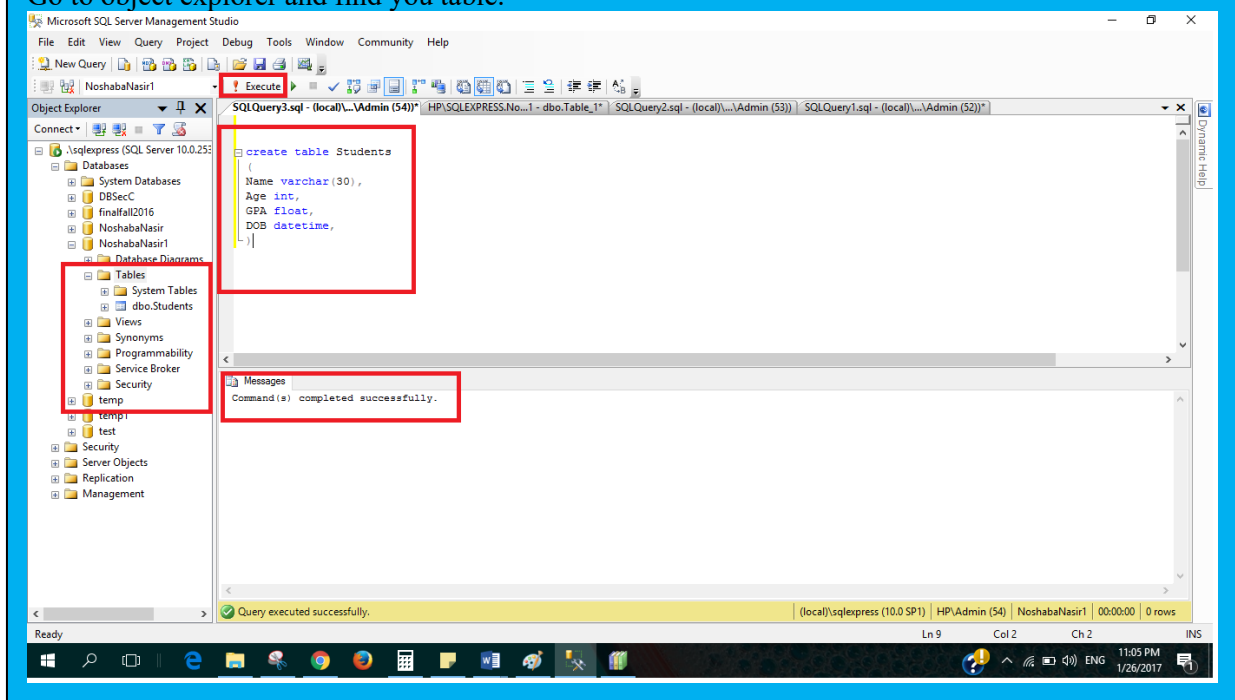
Also called DDL includes creating objects such as tables, adding constraints of tables, such as FK PK unique constraints, in this section you will learn how to create tables and add PK, FK constraints using Queries.

Step 1: Create table:

Following is the syntax of table creating query

```
createtable [tablename]
(
[Column1Name] datatype null/Not null,
[Column2Name] datatype null/not null,
[Column2Name] datatype null/not null
)
*null is default
```

Create the table Students using the query as shown in following figure, Execute, make sure the message shows that the command is successful.
Go to object explorer and find you table.



Step 2: Altering table Schema

Adding new column to existing table

```
altertable <TableName> add <NewColumn> <Datatype>
```

Try this query

```
altertable Students add Address varchar (50)
```

Drop existing Column from existing table

```
altertable <TableName> dropcolumn <ColumnName>
```

Try this query

```
altertable Students dropcolumn [Address]
```

Step 3: Add Primary key Constraint.

One way to create PK is to add it whilst creating table using the following syntax

```
createtable [tablename]
(
[Column1Name] datatype primary key,
[Column2Name] datatype,
[Column2Name] datatype
)
```

OR if there is composite key

```
createtable [tablename]
(
[Column1Name] datatype,
[Column2Name] datatype,
[Column2Name] datatype,
Primarykey ([Column1Name], [Column2Name])
)
```

Try the following query, see in object explorer

```
create table Students1
(
RollNo int not null primary key,
Name varchar(30),
Age int,
GPA float,
DOB datetime,
)
```

Primary key should be not null

Try the following query, see in object explorer

```
create table Students2
(
batchNo int,
serialNo int,
Name varchar(30),
Age int,
GPA float,
DOB datetime,
Primary key (batchNo, serialNo)
)
```

Other way is to add PK constraint after creating the table, by using ALTER query,

```
altertable [tableName] addconstraint [keyConstraintName] Primarykey (Column1,
column2, column3)
```

- ▶ ALTER TABLE tblename
ADD PRIMARY KEY (ATT);
- ▶ ALTER TABLE tblename
ADD CONSTRAINT PK_NAME PRIMARY KEY (ATT1,ATT2);

Check The name of the constraints

```
SELECT
table_name,
constraint_type,
constraint_name
FROM information_schema.table_constraints
WHERE table_name = 'name of the table';
```

Try the following, PK columns should be not null

```
create table Students3
(
  RollNo int not null primary key ,
  Name varchar(30),
  Age int,
  GPA float,
  DOB datetime,
)

alter table Students3 add constraint Primarykey Primary key (RollNo)
```

How to see the schema of your table

Try this to see the schema of your table

```
sp_help<tableName>
```

This will give you information about columns, their datatype and all the constraints on the table.

Step 4: Add foreign key constraint to tables

Before creating FK constraint Make sure that the referred table and its referred columns are created.

Add FK whilst creating table.

Use the following syntax as given.

```
createtable [tablename]
(
  [Column1Name] datatype Null/Not Null,
  [Column2Name] datatype Null/Not Null,
  [ReferencingColumn] datatype Null/Not Null
  FOREIGN KEY REFERENCES referedtable(referedColum)
)
```

*no action is default

Try the following, and see in object explorer.

If you want to create Foreign keys through script while creating the tables then use the scripts shown

Note : the Referenced table should be created before the referring table

This line will set FK constraint

```
CREATE TABLE dbo.Staff_One(  
    staffID [int] Primary Key NOT NULL,  
    staffName [varchar] (50) NULL,  
    staffRole [varchar] (50) NULL  
)  
  
CREATE TABLE dbo.School_one(  
    schID [int] Primary Key NOT NULL,  
    schName [varchar] (50) NULL,  
    schdeanID [int] FOREIGN KEY References Staff2 (staffid) ON DELETE NO ACTION ON UPDATE NO ACTION NOT NULL  
)
```

Referencing Column

Referenced Table

Referenced Column of Referenced table

Insertion and Update Specifications

FK can also be added after the referencing and referred tables have been created.

Try the following and see in object explorer

The screenshot shows a SQL Server Enterprise Manager window with the following SQL script:

```

CREATE TABLE dbo.Staff_two(
  staffID [int] Primary Key NOT NULL,
  staffName [varchar] (50) NULL,
  staffRole [varchar] (50) NULL
)

CREATE TABLE dbo.School_two(
  schID [int] Primary Key NOT NULL,
  schName [varchar] (50) NULL,
  schdeanID [int] NOT NULL
)

Alter table dbo.School_two
Add Constraint FK_School_Staff_two
FOREIGN KEY (schdeanID) References Staff_two(staffid) ON DELETE Cascade ON UPDATE NO ACT
  
```

Handwritten annotations on the screenshot include:

- 1. Create the tables first** (pointing to the CREATE TABLE statements)
- 2. Now Alter the table School_Two and add FK constraint** (pointing to the ALTER TABLE statement)
- Name of FK** (pointing to the constraint name FK_School_Staff_two)
- Referencing Column** (pointing to schdeanID)
- Referenced table and Col** (pointing to Staff_two(staffid))
- Insert and Update Specs.** (pointing to ON DELETE Cascade ON UPDATE NO ACT)

A yellow box on the right contains the text: "Now if you have already created the tables and you want to add the FK constraints, you can use the script as shown".

At the bottom, the Messages pane shows: "command(s) completed successfully."

6. Data Modification Language

Step 1: Insert rows into table

Syntax

-- let there be N columns in a table

INSERT INTO <tableName>

values

(Column1Value, Column2Value, Column3Value, ..., ColumnNValue), --row 1

(Column1Value, Column2Value, Column3Value, ..., ColumnNValue), --row 2

(Column1Value, Column2Value, Column3Value, ..., ColumnNValue) --row 3

Or

INSERT INTO <tableName> (ColumnX, ColumnY) -- List of Columns

values

(ColumnXValue, ColumnYValue), --Row 1 list of Corresponding column values

(ColumnXValue, ColumnYValue), --Row 2

(ColumnXValue, ColumnYValue) --Row 3

Try the following

```

INSERT INTO [Students1] ([RollNo], [Name], [Age], [GPA], [DOB])
VALUES (13, 'Ahmed', '20', 2.4, '1/1/1990')
GO
  
```

Try the following

```
INSERT INTO [Students1] ([RollNo], [Name], [GPA])  
VALUES      (13, 'Ahmed', 2.4)  
GO
```

Step 2: To see the data from you table use the following table

`Select * from tableName`

Try the following and see the results

```
Select * from Students1
```

Results				
Messages				
RollNo	Name	Age	GPA	DOB
13	Ahmed	20	2.4	1990-01-01 00:00:00.000
13	Ahmed	NULL	2.4	NULL

Step 3: Delete rows from the table

`Delete from <tableName>`

`Where <conditions>`

Try the following and see the message and data in table using select query

```
Delete from Students1
Where Age=20
```

Messages				
row(s) affected)				

Step 4: Delete all the data from the table

`Delete from <tableName> --as there is no where condition all rows will be deleted`

Or

`Truncate table <tableName>`

Try these two commands after adding some rows to the table and see the results.

Step 5: Updating the rows

```
Update tableName
set ColumnA=<NewValue>,
ColumnB=<NewValue>,
where <Conditions>
```

See the data in table before and after trying the following query

```
Update Students1  
set Name='Ali Ahmed'  
where Name='Ahmed'
```

Exercise

Will be given in lab

Appendix

Commentsis SQL

Comments in SQL server start with two dashes as shown below, in green color

```
--This query create student table  
CREATE TABLE students  
(  
  id INT,  
  fullName varchar(40)|  
)
```

Where Conditions

Or
And
Not
In

Sub queries

In
All
Any
Some
Exists

How to see details of certain table

```
sp_help tableName
```

CSC270-Database Systems I

Lecture-2 (Lab)

(Spring 2026)

Create table

► CREATE TABLE Staff (
 staffNo int,
 fName varchar(255),
 lName varchar(255),
 position varchar(255),
 gender char ,
 DOB date,
 salary money,
);

Agenda

- ▶ SQL Constraints

SQL Constraints

- ▶ Used to specify rules for data in a table.
- ▶ Specify at the time of table creation
- ▶ Can alter if there is any change

SQL Constraints

- ▶ NOT NULL - Ensures that a column cannot have a NULL value
- ▶ UNIQUE - Ensures that all values in a column are different
- ▶ PRIMARY KEY - A combination of a NOT NULL and UNIQUE. Uniquely identifies each row in a table
- ▶ DEFAULT - Sets a default value for a column when no value is specified
- ▶ FOREIGN KEY - Uniquely identifies a row/record in another table
- ▶ CHECK - Ensures that all values in a column satisfies a specific condition
- ▶ INDEX - Used to create and retrieve data from the database very quickly

SQL NOT NULL Constraint

- ▶ CREATE TABLE Staff (
 staffNo int NOT NULL,
 fName varchar(255),
 lName varchar(255),
 position varchar(255),
 gender char ,
 DOB date,
 salary money
);
- ▶ ALTER TABLE Staff
 MODIFY gender char NOT NULL;

SQL UNIQUE Constraint

► SQL Server Unique Constraint

► CREATE TABLE Staff (
 staffNo int NOT NULL UNIQUE,
 fName varchar(255),
 lName varchar(255),
 position varchar(255),
 gender char ,
 DOB date,
 salary money,
);

► MySQL Unique Constraint

► CREATE TABLE Staff (
 staffNo int NOT NULL,
 fName varchar(255),
 lName varchar(255),
 position varchar(255),
 gender char ,
 DOB date,
 salary money,
 UNIQUE (StaffNo)
);



To Name a Unique Constraint

```
► CREATE TABLE Staff (  
    staffNo int NOT NULL UNIQUE,  
    fName varchar(255),  
    lName varchar(255),  
    position varchar(255),  
    gender char ,  
    DOB date,  
    salary money,  
    CONSTRAINT UC_Staff UNIQUE (StaffNo)  
);
```



SQL UNIQUE Constraint on ALTER TABLE

► For Single Column

- ALTER TABLE Staff
ADD UNIQUE (staffNo);

► For Multiple Column

- ALTER TABLE Staff
ADD CONSTRAINT UC_Staff UNIQUE (staffNo,fName);

DROP a UNIQUE Constraint

► SQL Server

► ALTER TABLE Staff
DROP CONSTRAINT UC_Staff;

► MySQL

► ALTER TABLE Staff
DROP INDEX UC_Staff;

SQL PRIMARY KEY Constraint

► MS SQL Server

```
► CREATE TABLE Staff (  
    staffNo int NOT NULL PRIMARY KEY,  
    fName varchar(255),  
    lName varchar(255),  
    position varchar(255),  
    gender char ,  
    DOB date,  
    salary money  
);
```

► MySQL

```
► CREATE TABLE Staff (  
    staffNo int NOT NULL,  
    fName varchar(255),  
    lName varchar(255),  
    position varchar(255),  
    gender char ,  
    DOB date,  
    salary money,  
    PRIMARY KEY (staffNo)  
);
```

PRIMARY KEY constraint on multiple columns

```
► CREATE TABLE Staff (  
    staffNo int NOT NULL,  
    fName varchar(255),  
    lName varchar(255),  
    position varchar(255),  
    gender char ,  
    DOB date,  
    salary money,  
    CONSTRAINT PK_Staff PRIMARY KEY (staffNo,fName)  
);
```

SQL PRIMARY KEY on ALTER TABLE

- ▶ ALTER TABLE Staff
ADD PRIMARY KEY (staffNo);
- ▶ ALTER TABLE Staff
ADD CONSTRAINT PK_Staff PRIMARY KEY (staffNo,fName);

DROP a PRIMARY KEY Constraint

► MS SQL SERVER

- ALTER TABLE Staff
DROP CONSTRAINT PK_Staff;

► MySQL

- ALTER TABLE Staff
DROP PRIMARY KEY;

SQL DEFAULT Constraint

```
► CREATE TABLE Staff (  
    staffNo int NOT NULL PRIMARY KEY,  
    fName varchar(255),  
    lName varchar(255),  
    position varchar(255) DEFAULT 'Supervisor',  
    gender char ,  
    DOB date,  
    salary money  
);
```

```
CREATE TABLE Orders (  
    ID int NOT NULL,  
    OrderNumber int NOT NULL,  
    OrderDate date DEFAULT GETDATE()  
);
```

SQL DEFAULT on ALTER TABLE

- ▶ **SQL Server:**

- ▶ ALTER TABLE Staff
ADD CONSTRAINT df_Position DEFAULT 'Supervisor' FOR Position;

- ▶ **MySQL:**

- ▶ ALTER TABLE Staff
ALTER Position SET DEFAULT 'Supervisor';

DROP a DEFAULT Constraint

- ▶ **SQL Server:**

- ▶ ALTER TABLE Staff
ALTER COLUMN Position DROP DEFAULT;

- ▶ **MySQL:**

- ▶ ALTER TABLE Staff
ALTER Position DROP DEFAULT;

Summary

▶ SQL Constraints

- ▶ NOT NULL
- ▶ Primary Key
- ▶ Unique
- ▶ Default

Previous Lecture Review

▶ SQL Constraints

- ▶ NOT NULL
- ▶ Primary Key
- ▶ Unique
- ▶ Default

Create table

► CREATE TABLE Staff (
 staffNo int,
 fName varchar(255),
 lName varchar(255),
 position varchar(255),
 gender char ,
 DOB date,
 salary money,
);

Agenda

▶ SQL Constraints

- ▶ CHECK

- ▶ INDEX

- ▶ FOREIGN KEY

SQL Constraints

- ▶ CHECK - Ensures that all values in a column satisfies a specific condition
- ▶ INDEX - Used to create and retrieve data from the database very quickly
- FOREIGN KEY - Uniquely identifies a row/record in another table

SQL CHECK on CREATE TABLE

► SQL Server CHECK Constraint

► CREATE TABLE Staff (
 staffNo int NOT NULL,
 fName varchar(255),
 lName varchar(255),
 position varchar(255),
 DOB date,
 salary money CHECK (salary>=17000)
);

► MySQL CHECK Constraint

► CREATE TABLE Staff (
 staffNo int NOT NULL,
 fName varchar(255),
 lName varchar(255),
 position varchar(255),
 DOB date,
 salary money,
 CHECK (salary>=17000)
);

CHECK constraint on multiple columns

```
► CREATE TABLE Staff (  
    staffNo int NOT NULL UNIQUE,  
    fName varchar(255),  
    lName varchar(255),  
    position varchar(255),  
    gender char ,  
    DOB date,  
    salary money,  
    CONSTRAINT CHK_Staff CHECK (salary>=17000 AND position='Supervisor')  
);
```

SQL CHECK on ALTER TABLE

- ▶ For Single Column

- ▶ ALTER TABLE staff
ADD CHECK (salary>=17000);

- ▶ ALTER TABLE staff ADD CHECK (position='supervisor' or position='Manager');

- ▶ For Multiple Column

- ▶ ALTER TABLE staff
ADD CONSTRAINT CHK_staff CHECK (salary>=17000 AND position='Supervisor');

DROP a CHECK Constraint

► SQL Server

► ALTER TABLE Staff
DROP CONSTRAINT CHK_staff;

► MySQL

► ALTER TABLE Staff
DROP CHECK CHK_staff;

SQL CREATE INDEX Statement

- ▶ CREATE INDEX *index_name*
ON *table_name* (*column1*, *column2*, ...);
- ▶ CREATE UNIQUE INDEX *index_name*
ON *table_name* (*column1*, *column2*, ...);

SQL CREATE INDEX Statement

- ▶ `CREATE INDEX idx_lname
ON staff (lname);`
- ▶ `CREATE INDEX idx_sname
ON staff (lname, fname);`

DROP INDEX Statement

- ▶ **SQL Server:**

- ▶ `DROP INDEX staff. idx_sname;`

- ▶ **MySQL:**

- ▶ `ALTER TABLE staff
DROP INDEX idx_sname;`

Foreign Key Concept

Branch

branchNo	street	city	postcode
B005	22 Deer Rd	London	SW1 4EH
B007	16 Argyll St	Aberdeen	AB2 3SU
B003	163 Main St	Glasgow	G11 9QX
B004	32 Manse Rd	Bristol	BS99 1NZ
B002	56 Clover Dr	London	NW10 6EU

Staff

staffNo	fName	IName	position	sex	DOB	salary	branchNo
SL21	John	White	Manager	M	1-Oct-45	30000	B005
SG37	Ann	Beech	Assistant	F	10-Nov-60	12000	B003
SG14	David	Ford	Supervisor	M	24-Mar-58	18000	B003
SA9	Mary	Howe	Assistant	F	19-Feb-70	9000	B007
SG5	Susan	Brand	Manager	F	3-Jun-40	24000	B003
SL41	Julie	Lee	Assistant	F	13-Jun-65	9000	B005

SQL FOREIGN KEY Constraint

► MS SQL Server

```
► CREATE TABLE Branch (  
    branchNo varchar(10) NOT NULL  
    PRIMARY KEY,  
    street varchar(255),  
    city varchar(255),  
    postCode varchar(255),  
);
```

```
► CREATE TABLE Staff (  
    staffNo int NOT NULL PRIMARY KEY,  
    fName varchar(255),  
    lName varchar(255),  
    position varchar(255),  
    sex char ,  
    DOB date,  
    salary money,  
    branchNo varchar(10) ,  
    CONSTRAINT FK_BranchStaff FOREIGN KEY (branchNo)  
    REFERENCES Branch(branchNo)  
);
```

SQL FOREIGN KEY on ALTER TABLE

- ▶ ALTER TABLE Staff
ADD FOREIGN KEY (branchNo) REFERENCES Branch(branchNo);
- ▶ **Add Foreign key Constraint with Constraint Name**
- ▶ ALTER TABLE Staff
ADD CONSTRAINT FK_BranchStaff
FOREIGN KEY (branchNo) REFERENCES Branch(branchNo);

DROP a FOREIGN KEY Constraint

► MS SQL SERVER

► ALTER TABLE Staff
DROP CONSTRAINT FK_BranchStaff;

► MySQL

► ALTER TABLE Staff
DROP FOREIGN KEY FK_BranchStaff;

Bonus

SQL AUTO INCREMENT Field

- ▶ CREATE TABLE Persons (
 Personid int IDENTITY(1,1) PRIMARY KEY,
 LastName varchar(255) NOT NULL,
 FirstName varchar(255),
 Age int
);
- ▶ INSERT INTO Persons (FirstName, LastName, age)
VALUES ('Lars','Monsen',30),('Malinda','John',32);
- ▶ Select * from Persons
- ▶ Select Personid,FirstName,LastName,Age from Persons

Personid	First Name	Last Name	Age
1	Lars	Monsen	30
2	Malinda	John	32

SQL AUTO INCREMENT Field For MySQL

- ▶ CREATE TABLE Persons (
 Personid int NOT NULL AUTO_INCREMENT,
 LastName varchar(255) NOT NULL,
 FirstName varchar(255),
 Age int,
 PRIMARY KEY (Personid)
);
- ▶ ALTER TABLE Persons AUTO_INCREMENT=100;

Summary

▶ SQL Constraints

- ▶ CHECK
- ▶ INDEX
- ▶ FOREIGN KEY

CSC270-Database Systems I (Lab) (SPRING 2026)

Create table

► CREATE TABLE Branch (
 branchNo varchar(10) NOT NULL PRIMARY KEY,
 street varchar(50),
 city varchar(50),
 postcode varchar(20),
);

```
CREATE TABLE Staff(  
    staffNo VARCHAR(15) NOT NULL,  
    fName VARCHAR(15) NOT NULL,  
    lName VARCHAR(15) NOT NULL,  
    position VARCHAR(15) NOT NULL,  
    sex VARCHAR(15) NOT NULL,  
    DOB DATE NOT NULL,  
    salary INT NOT NULL,  
    branchNo VARCHAR(10) NULL,  
    CONSTRAINT pk_staffs PRIMARY KEY(staffNo),  
    CONSTRAINT fk_staffs FOREIGN KEY(branchNO) REFERENCES Branch1(branchNo)  
);
```

Branch

	branchNo	street	city	postCode
1	B002	56 Clover Dr	London	NW10 6EU
2	B003	163 Main St	Glasgow	G11 9QX
3	B004	32 Manse Rd	Bristol	BS99 1NZ
4	B005	22 Deer Rd	London	SW1 4EH
5	B007	16 Argyll St	Aberdeen	AB2 3SU

Staff

	staffNo	fName	lName	position	sex	DOB	salary	branchNo
1	SG14	David	Ford	Supervisor	M	1958-11-24	18000	B003
2	SG37	Ann	Beech	Assistant	F	1960-10-11	12000	B003
3	SG5	Susan	Brand	Manager	F	1940-06-03	24000	B003
4	SL21	John	White	Manager	M	1945-10-01	30000	B005
5	SL41	Julie	Lee	Assistant	F	1965-06-13	9000	B005
6	SA9	Mary	Howe	Assistant	F	1970-02-19	9000	B007

Agenda

- Where Clause
- Operators used in SQL
- Logical Operators

Where Clause

- ▶ Used to filter the records
- ▶ Used with select statement
- ▶ Used to update data of records
- ▶ Used to delete records
- ▶ Can also used to join more than one table

Operators used in SQL

Operator	Description
=	Equal
>	Greater than
<	Less than
>=	Greater than or equal
<=	Less than or equal
<>	Not equal. Note: In some versions of SQL this operator may be written as !=
BETWEEN	Between a certain range
LIKE	Search for a pattern
IN	To specify multiple possible values for a column

Where clause

- ▶ `SELECT column1, column2, ...`
`FROM table_name`
`WHERE condition;`

- ▶ `Select * from branch where branchNo = 'B003'`

	branchNo	street	city	postCode
1	B003	163 Main St	Glasgow	G11 9QX

- ▶ `Select * from staff where salary = 30000`

	staffNo	fName	lName	position	sex	DOB	salary	branchNo
1	SL21	John	White	Manager	M	1945-10-01	30000	B005

Use of different operators within condition

- Select * from staff where salary >= 18000

	staffNo	fName	lName	position	sex	DOB	salary	branchNo
1	SG14	David	Ford	Supervisor	M	1958-11-24	18000	B003
2	SG5	Susan	Brand	Manager	F	1940-06-03	24000	B003
3	SL21	John	White	Manager	M	1945-10-01	30000	B005

All values except...18000

- Select * from staff where salary <> 18000

	staffNo	fName	lName	position	sex	DOB	salary	branchNo
1	SA9	Mary	Howe	Assistant	F	1970-02-19	9000	B007
2	SG37	Ann	Beech	Assistant	F	1960-10-11	12000	B003
3	SG5	Susan	Brand	Manager	F	1940-06-03	24000	B003
4	SL21	John	White	Manager	M	1945-10-01	30000	B005
5	SL41	Julie	Lee	Assistant	F	1965-06-13	9000	B005

Use of between in where clause

- Select * from staff where salary BETWEEN 12000 AND 18000

	staffNo	fName	lName	position	sex	DOB	salary	branchNo
1	SG14	David	Ford	Supervisor	M	1958-11-24	18000	B003
2	SG37	Ann	Beech	Assistant	F	1960-10-11	12000	B003
3	SG5	Susan	Brand	Manager	F	1940-06-03	24000	B003

- Select * from staff where salary NOT BETWEEN 12000 AND 18000

	staffNo	fName	lName	position	sex	DOB	salary	branchNo
1	SA9	Mary	Howe	Assistant	F	1970-02-19	9000	B007
2	SL21	John	White	Manager	M	1945-10-01	30000	B005
3	SL41	Julie	Lee	Assistant	F	1965-06-13	9000	B005

Use of IN Operator

- Select * from staff where position IN ('MANAGER','SUPERVISOR')

	staffNo	fName	lName	position	sex	DOB	salary	branchNo
1	SG14	David	Ford	Supervisor	M	1958-11-24	18000	B003
2	SG5	Susan	Brand	Manager	F	1940-06-03	24000	B003
3	SL21	John	White	Manager	M	1945-10-01	30000	B005

- Select * from staff where position NOT IN ('MANAGER','SUPERVISOR')

	staffNo	fName	lName	position	sex	DOB	salary	branchNo
1	SA9	Mary	Howe	Assistant	F	1970-02-19	9000	B007
2	SG37	Ann	Beech	Assistant	F	1960-10-11	12000	B003
3	SL41	Julie	Lee	Assistant	F	1965-06-13	9000	B005

LIKE operator and wild card (% , _)

- ▶ select * from branch where postcode LIKE 'sw1%'

	branchNo	street	city	postCode
1	B005	22 Deer Rd	London	SW1 4EH

- ▶ select * from branch where postcode LIKE 'AB2 3S_'

	branchNo	street	city	postCode
1	B007	16 Argyll St	Aberdeen	AB2 3SU

Logical Operators

OR, AND, NOT

Logical Operator OR

- ▶ `SELECT column1, column2, ...
FROM table_name
WHERE condition1 OR condition2 OR condition3 ...;`
- ▶ We need the branch details for London, Bristol and Glasgow city.
- ▶ `Select * from branch where city = 'London' OR city ='Bristol' OR city = 'Glasgow'`

	branchNo	street	city	postCode
1	B002	56 Clover Dr	London	NW10 6EU
2	B003	163 Main St	Glasgow	G11 9QX
3	B004	32 Manse Rd	Bristol	BS99 1NZ
4	B005	22 Deer Rd	London	SW1 4EH

Logical Operator AND

- ▶ `SELECT column1, column2, ...
FROM table_name
WHERE condition1 AND condition2 AND condition3 ...;`
- ▶ `Select * From Branch WHERE city = 'London' AND branchNo= 'B002' AND postcode LIKE 'NW10%'`

	branchNo	street	city	postCode
1	B002	56 Clover Dr	London	NW10 6EU

Logical Operator NOT

- ▶ `SELECT column1, column2, ...
FROM table_name
WHERE NOT condition;`
- ▶ `Select * From Branch WHERE NOT city = 'London'`
- ▶ `Select * From Branch WHERE NOT city = 'London' AND NOT city = 'Bristol'`

	branchNo	street	city	postCode
1	B003	163 Main St	Glasgow	G11 9QX
2	B007	16 Argyll St	Aberdeen	AB2 3SU

IS NULL , IS NOT NULL

- ▶ `SELECT column_names
FROM table_name
WHERE column_name IS NULL;`
- ▶ `Select * From staff WHERE branchNo IS NULL`

	staffNo	fName	lName	position	sex	DOB	salary	branchNo
1	SA9	Mary	Howe	Assistant	F	1970-02-19	9000	NULL

- ▶ `Select * From staff WHERE branchNo IS NOT NULL`

	staffNo	fName	lName	position	sex	DOB	salary	branchNo
1	SG14	David	Ford	Supervisor	M	1958-11-24	18000	B003
2	SG37	Ann	Beech	Assistant	F	1960-10-11	12000	B003
3	SG5	Susan	Brand	Manager	F	1940-06-03	24000	B003
4	SL21	John	White	Manager	M	1945-10-01	30000	B005
5	SL41	Julie	Lee	Assistant	F	1965-06-13	9000	B005

Summary

- Where Clause
- Operators used in SQL
- Logical Operators

Previous Lecture Review

- SQL Update
- SQL Delete
- SQL Select Top
- SQL ORDER BY
- SQL Aliases
- SQL MIN() and MAX() Functions
- SQL COUNT(), AVG() and SUM() Functions

Branch

	branchNo	street	city	postCode
1	B002	56 Clover Dr	London	NW10 6EU
2	B003	163 Main St	Glasgow	G11 9QX
3	B004	32 Manse Rd	Bristol	BS99 1NZ
4	B005	22 Deer Rd	London	SW1 4EH
5	B007	16 Argyll St	Aberdeen	AB2 3SU

Staff

	staffNo	fName	lName	position	sex	DOB	salary	branchNo
1	SG14	David	Ford	Supervisor	M	1958-11-24	18000	B003
2	SG37	Ann	Beech	Assistant	F	1960-10-11	12000	B003
3	SG5	Susan	Brand	Manager	F	1940-06-03	24000	B003
4	SL21	John	White	Manager	M	1945-10-01	30000	B005
5	SL41	Julie	Lee	Assistant	F	1965-06-13	9000	B005
6	SA9	Mary	Howe	Assistant	F	1970-02-19	9000	B007

Agenda

- SQL Group By Clause
- SQL Having Clause

GROUP BY Syntax

- ▶ Groups rows that have the same values into summary rows
 - ▶ Find the number of staff members in each branch
 - ▶ It is often used with aggregate functions to group the result-set by one or more columns.
 - ▶ Used COUNT, MAX, MIN, SUM, AVG functions.
- ▶

```
SELECT column_name(s)
FROM table_name
WHERE condition
GROUP BY column_name(s)
ORDER BY column_name(s);
```

- Show branch wise data from staff table?
- Select * from staff group by branchno
- Select * from staff order by branchno

staffNo	fName	lName	position	sex	DOB	salary	branchNo
SG14	David	Ford	Supervisor	M	1958-11-24	18000	B003
SG37	Ann	Beech	Assistant	F	1960-10-11	12000	B003
SG5	Susan	Brand	Manager	F	1940-06-03	24000	B003
SL21	John	White	Manager	M	1945-10-01	30000	B005
SL41	Julie	Lee	Assistant	F	1965-06-13	9000	B005
SA9	Mary	Howe	Assistant	F	1970-02-19	9000	B007

Use of Group by with Count function

- ▶ Count how many employees in each branch?
- ▶ `SELECT branchno, COUNT(staffno) AS [Staff in each Branch]`
`FROM staff`
`GROUP BY branchno;`

branchno	Staff in each Branch
B003	3
B005	2
B007	1

Group By with ORDER BY Clause

- ▶ SELECT branchno, COUNT(staffno) AS [Staff in each Branch]
FROM staff
GROUP BY branchno
ORDER BY COUNT(staffno) ASC;

branchno	Staff in each Branch
B007	1
B005	2
B003	3

- ▶ SELECT branchno, COUNT(staffno) AS [Staff in each Branch]
FROM staff
GROUP BY branchno
ORDER BY branchno DESC;

- Can you find out the average salary of Assistants in each branch?

branchno	Average salary of Assistants in each Branch
B007	9000
B005	9000
B003	12000

- ```
SELECT branchno, Avg(salary) AS [Average salary of Assistants in each Branch]
FROM staff
Where position = 'assistant'
GROUP BY branchno
ORDER BY branchno DESC;
```

# Can you generate a report for branches where more than two employees are working?

- ▶ `select branchNo, count(staffno) As [Total Employee]`  
    `From staff`  
    `where count(staffno) > 2`  
    `Group by branchno`

# The SQL HAVING Clause

- ▶ The HAVING clause was added to SQL because the WHERE keyword could not be used with aggregate functions.
- ▶ Applied condition on group data

▶  
`SELECT column_name(s)`  
`FROM table_name`  
`WHERE condition`  
`GROUP BY column_name(s)`  
`HAVING condition`  
`ORDER BY column_name(s);`

# Can we find out the branch number with more than two employees

- `select branchNo, count(staffno) As [Total Employee]`  
`From staff`  
`Group by branchno;`  
`Having count(staffno) > 2`

| branchNo | Total Employee |
|----------|----------------|
| B003     | 3              |

► select branchNo, count(staffno) As [Total Employee], Avg(salary) AS [Average Salary]  
From staff  
Where salary >= 9000  
Group by branchno  
Having count(staffno) > 1  
Order By count(staffno) DESC;

- ▶ select branchNo, count(staffno) As [Total Employee], Avg(salary) AS [Average Salary]  
From staff  
Where salary > 10000  
Group by branchno  
Having count(staffno) > 1;

| branchNo | Total Employee | Average Salary |
|----------|----------------|----------------|
| B003     | 3              | 18000          |

Is the Result correct?  
If yes How?  
If Not Why?

# SQL CREATE VIEW Statement

- ▶ Virtual table based on the result-set of an SQL statement.
- ▶ The fields in a view are fields from one or more base/real tables in the database.
- ▶ Can add SQL functions, WHERE, JOIN statements and sub-query.
- ▶ Fetch data from many tables to a view and present the data as if the data were coming from one single table.
- ▶ Security Purpose
- ▶ Reporting Services

# SQL CREATE VIEW

- ▶ CREATE VIEW *view\_name* AS  
SELECT *column1*, *column2*, ...  
FROM *table\_name*  
WHERE *condition*;
- ▶ CREATE VIEW [Branch\_B003] AS  
SELECT staffno, fname, salary  
FROM staff  
WHERE branchno = 'B003';
- ▶ select \* from branch\_B003

| staffno | fname | salary |
|---------|-------|--------|
| SG14    | David | 18000  |
| SG37    | Ann   | 12000  |
| SG5     | Susan | 24000  |

# SQL CREATE OR REPLACE VIEW

- ▶ CREATE OR ALTER VIEW [Branch\_B003] AS  
SELECT staffno, fname, lname, salary  
FROM staff  
WHERE branchno = 'B003';
- ▶ select \* from branch\_B003

| staffno | fname | lname | salary |
|---------|-------|-------|--------|
| SG14    | David | Ford  | 18000  |
| SG37    | Ann   | Beech | 12000  |
| SG5     | Susan | Brand | 24000  |

# Drop View

► DROP VIEW [branch\_b003];

# Summary

- SQL Group By Clause
- SQL Having Clause
- SQL Views



# COMSATS University Islamabad, Lahore Campus

## Department of Computer Science

### Graded Lab Spring 2026 (Lab Assignment 1)

|                    |                    |               |                |               |        |
|--------------------|--------------------|---------------|----------------|---------------|--------|
| Course Title:      | Database Systems I | Course Code:  | CSC270         | Credit Hours: | 4(3,1) |
| Course Instructor: | Mahrukh Saleem     | Program Name: | BS CS          |               |        |
| Semester:          | 4 <sup>th</sup>    | Batch:        | FA24           | Section:      | C      |
| Date:              | 14-03-2026         |               |                |               |        |
| Time Allowed:      | 1 Hour             |               | Maximum Marks: | 12            |        |
| Student's Name:    |                    |               | Reg. No.       |               |        |

#### Important Instructions / Guidelines:

- Plan and manage your time accordingly.
- You are required to submit the .sql file on google classroom.
- Name your file after your registration number. For example: FA24-BCS-000

**Question 1:** CLO: <5>; Bloom Taxonomy Level: <Applying> [Apply data processing operations on both relational and non-relational DBMS]

Create the following tables with reasonable constraints and insert given values into relevant tables

#### Branch

| branchNo | street       | city     | postcode |
|----------|--------------|----------|----------|
| B005     | 22 Deer Rd   | London   | SW1 4EH  |
| B007     | 16 Argyll St | Aberdeen | AB2 3SU  |
| B003     | 163 Main St  | Glasgow  | G11 9QX  |
| B004     | 32 Manse Rd  | Bristol  | BS99 1NZ |
| B002     | 56 Clover Dr | London   | NW10 6EU |

#### Staff

| staffNo | fName | lName | position   | sex | DOB       | salary | branchNo |
|---------|-------|-------|------------|-----|-----------|--------|----------|
| SL21    | John  | White | Manager    | M   | 1-Oct-45  | 30000  | B005     |
| SG37    | Ann   | Beech | Assistant  | F   | 10-Nov-60 | 12000  | B003     |
| SG14    | David | Ford  | Supervisor | M   | 24-Mar-58 | 18000  | B003     |
| SA9     | Mary  | Howe  | Assistant  | F   | 19-Feb-70 | 9000   | B007     |
| SG5     | Susan | Brand | Manager    | F   | 3-Jun-40  | 24000  | B003     |
| SL41    | Julie | Lee   | Assistant  | F   | 13-Jun-65 | 9000   | B005     |

#### PrivateOwner

| ownerNo | fName | lName  | address                       | telNo         | eMail             |
|---------|-------|--------|-------------------------------|---------------|-------------------|
| CO46    | Joe   | Keogh  | 2 Fergus Dr, Aberdeen AB2 7SX | 01224-861212  | jkeogh@lhh.com    |
| CO87    | Carol | Farrel | 6 Achray St, Glasgow G32 9DX  | 0141-357-7419 | cfarrel@gmail.com |
| CO40    | Tina  | Murphy | 63 Well St, Glasgow G42       | 0141-943-1728 | tinam@hotmail.com |
| CO93    | Tony  | Shaw   | 12 Park Pl, Glasgow G4 0QR    | 0141-225-7025 | tony.shaw@ark.com |



# COMSATS University Islamabad, Lahore Campus

## Department of Computer Science

PropertyForRent

| propertyNo | street        | city     | postcode | type  | rooms | rent | ownerNo | staffNo | branchNo |
|------------|---------------|----------|----------|-------|-------|------|---------|---------|----------|
| PA14       | 16 Holhead    | Aberdeen | AB7 5SU  | House | 6     | 650  | CO46    | SA9     | B007     |
| PL94       | 6 Argyll St   | London   | NW2      | Flat  | 4     | 400  | CO87    | SL41    | B005     |
| PG4        | 6 Lawrence St | Glasgow  | G11 9QX  | Flat  | 3     | 350  | CO40    |         | B003     |
| PG36       | 2 Manor Rd    | Glasgow  | G32 4QX  | Flat  | 3     | 375  | CO93    | SG37    | B003     |
| PG21       | 18 Dale Rd    | Glasgow  | G12      | House | 5     | 600  | CO87    | SG37    | B003     |
| PG16       | 5 Novar Dr    | Glasgow  | G12 9AX  | Flat  | 4     | 450  | CO93    | SG14    | B003     |

**Question: Answer the following queries**

- 1) Display the complete address of those house properties who have more than 4 rooms and rent less than 500
- 2) Display the full names of those owners whose property is in Glasgow
- 3) Display the staff numbers and names whose properties are located in Glasgow.
- 4) Display the full names(one column) of those Private owners who have hotmail email addresses.
- 5) List the owner name and property number of the property with minimum rent.

*Best of luck.*

# CSC270-Database Systems I

## Lab-04

### (Spring 2026)

Mahrukh Saleem

[mahrukhsaleem@cuilahore.edu.pk](mailto:mahrukhsaleem@cuilahore.edu.pk)

- ▶ CREATE TABLE Branch(
  - ▶ branchNo VARCHAR(15) NOT NULL,
  - ▶ street VARCHAR(20) NOT NULL,
  - ▶ city VARCHAR(20) NOT NULL,
  - ▶ postcode VARCHAR(15) NOT NULL,
  - ▶ CONSTRAINT pk\_branches PRIMARY KEY(branchNo)
- ▶ );
- ▶ CREATE TABLE Staff(
  - ▶ staffNo VARCHAR(10) NOT NULL
  - ▶ fName VARCHAR(15) NOT NULL,
  - ▶ lName VARCHAR(15) NOT NULL,
  - ▶ position VARCHAR(15) NOT NULL,
  - ▶ sex VARCHAR(15) NOT NULL,
  - ▶ DOB DATE NOT NULL,
  - ▶ salary INT(15) NOT NULL,
  - ▶ branchNo VARCHAR(15) NOT NULL,
  - ▶ CONSTRAINT pk\_staffs PRIMARY KEY(staffNo),
  - ▶ CONSTRAINT fk\_staffs FOREIGN KEY(branchNO) REFERENCES Branch(branchNo)
- ▶ );

► CREATE TABLE PrivateOwner(  
ownerNo VARCHAR(15) NOT NULL,  
fName VARCHAR(15) NOT NULL,  
lName VARCHAR(15) NULL,  
address VARCHAR(50) NOT NULL,  
telNo VARCHAR(15) NOT NULL,  
eMail VARCHAR(20) NOT NULL,  
CONSTRAINT pk\_privateOwners PRIMARY KEY(ownerNo)  
);

► CREATE TABLE PropertyForRent(  
propertyNo VARCHAR(15) NOT NULL,  
street VARCHAR(20) NOT NULL,  
city VARCHAR(20) NULL,  
postcode VARCHAR(15) NOT NULL,  
TYPE VARCHAR(15) NOT NULL,  
rooms INT NOT NULL,  
rent INT NOT NULL,  
ownerNo VARCHAR(15) NOT NULL,  
staffNo VARCHAR(10) NULL,  
branchNo VARCHAR(15) NOT NULL,  
CONSTRAINT pk\_propertyForRents PRIMARY KEY(propertyNo),  
CONSTRAINT fk\_propertyForRents FOREIGN KEY(ownerNO) REFERENCES PrivateOwner(ownerNo),  
CONSTRAINT fk\_property FOREIGN KEY(staffNo) REFERENCES Staff(staffNo),  
CONSTRAINT fk\_for FOREIGN KEY(branchNo) REFERENCES Branch(branchNo)  
);

- ▶ INSERT INTO Branch VALUES ('B005', '22 Deer Rd', 'London', 'SW1 4EH');
  - ▶ INSERT INTO Branch VALUES ('B007', '16 Argyll St', 'Aberdeen', 'AB2 3SU');
  - ▶ INSERT INTO Branch VALUES ('B003', '163 Main St', 'Glasgow', 'G11 9QX');
  - ▶ INSERT INTO Branch VALUES ('B004', '32 Manse Rd', 'Bristol', 'BS99 1NZ');
  - ▶ INSERT INTO Branch VALUES ('B002', '56 Clover Dr', 'London', 'NW10 6EU');
- 
- ▶ INSERT INTO Staff VALUES ('SL21', 'John', 'White', 'Manager', 'M', '1945-10-01', 30000, 'B005');
  - ▶ INSERT INTO Staff VALUES ('SG37', 'Ann', 'Beech', 'Assistant', 'F', '1960-10-11', 12000, 'B003');
  - ▶ INSERT INTO Staff VALUES ('SG14', 'David', 'Ford', 'Supervisor', 'M', '1958-11-24', 18000, 'B003');
  - ▶ INSERT INTO Staff VALUES ('SA9', 'Mary', 'Howe', 'Assistant', 'F', '1970-02-19', 9000, 'B007');
  - ▶ INSERT INTO Staff VALUES ('SG5', 'Susan', 'Brand', 'Manager', 'F', '1940-06-03', 24000, 'B003');
  - ▶ INSERT INTO Staff VALUES ('SL41', 'Julie', 'Lee', 'Assistant', 'F', '1965-06-13', 9000, 'B005');

- ▶ INSERT INTO PrivateOwner VALUES ('CO46', 'Joe', 'Keogh', '2 Fergus Dr, Aberdeen AB2 7SX', '01224-861212', 'jkeogh@lhh.com');
  - ▶ INSERT INTO PrivateOwner VALUES ('CO87', 'Carol', 'Farrel', '6 Achray St, Glasgow G32 9DX', '0141-357-7419', 'cfarrel@gmail.com');
  - ▶ INSERT INTO PrivateOwner VALUES ('CO40', 'Tina', 'Murphy', '63 Well St, Glasgow G42', '0141-943-1728', 'tinam@hotmail.com');
  - ▶ INSERT INTO PrivateOwner VALUES ('CO93', 'Tony', 'Shaw', '12 Park Pl, Glasgow G4 0QR', '0141-225-7025', 'tony.shaw@ark.com');
- 
- ▶ INSERT INTO PropertyForRent VALUES ('PA14', '16 Holhead', 'Aberdeen', 'AB7 5SU', 'House', 6, 650, 'CO46', 'SA9', 'B007');
  - ▶ INSERT INTO PropertyForRent VALUES ('PL94', '6 Argyll St', 'London', 'NW2', 'Flat', 4, 400, 'CO87', 'SL41', 'B005');
  - ▶ INSERT INTO PropertyForRent VALUES ('PG4', '6 Lawrence St', 'Glasgow', 'G11 9QX', 'Flat', 3, 350, 'CO40', NULL, 'B003');
  - ▶ INSERT INTO PropertyForRent VALUES ('PG36', '2 Manor Rd', 'Glasgow', 'G32 4QX', 'Flat', 3, 375, 'CO93', 'SG37', 'B003');
  - ▶ INSERT INTO PropertyForRent VALUES ('PG21', '18 Dale Rd', 'Glasgow', 'G12', 'House', 5, 600, 'CO87', 'SG37', 'B003');
  - ▶ INSERT INTO PropertyForRent VALUES ('PG16', '5 Novar Dr', 'Glasgow', 'G12 9AX', 'Flat', 4, 450, 'CO93', 'SG14', 'B003');

**Branch**

| branchNo | street       | city     | postcode |
|----------|--------------|----------|----------|
| B005     | 22 Deer Rd   | London   | SW1 4EH  |
| B007     | 16 Argyll St | Aberdeen | AB2 3SU  |
| B003     | 163 Main St  | Glasgow  | G11 9QX  |
| B004     | 32 Manse Rd  | Bristol  | BS99 1NZ |
| B002     | 56 Clover Dr | London   | NW10 6EU |

**Staff**

| staffNo | fName | lName | position   | sex | DOB       | salary | branchNo |
|---------|-------|-------|------------|-----|-----------|--------|----------|
| SL21    | John  | White | Manager    | M   | 1-Oct-45  | 30000  | B005     |
| SG37    | Ann   | Beech | Assistant  | F   | 10-Nov-60 | 12000  | B003     |
| SG14    | David | Ford  | Supervisor | M   | 24-Mar-58 | 18000  | B003     |
| SA9     | Mary  | Howe  | Assistant  | F   | 19-Feb-70 | 9000   | B007     |
| SG5     | Susan | Brand | Manager    | F   | 3-Jun-40  | 24000  | B003     |
| SL41    | Julie | Lee   | Assistant  | F   | 13-Jun-65 | 9000   | B005     |

**PrivateOwner**

| ownerNo | fName | lName  | address                       | telNo         | eMail             |
|---------|-------|--------|-------------------------------|---------------|-------------------|
| CO46    | Joe   | Keogh  | 2 Fergus Dr, Aberdeen AB2 7SX | 01224-861212  | jkeogh@lhh.com    |
| CO87    | Carol | Farrel | 6 Achray St, Glasgow G32 9DX  | 0141-357-7419 | cfarrel@gmail.com |
| CO40    | Tina  | Murphy | 63 Well St, Glasgow G42       | 0141-943-1728 | tinam@hotmail.com |
| CO93    | Tony  | Shaw   | 12 Park Pl, Glasgow G4 0QR    | 0141-225-7025 | tony.shaw@ark.com |

**PropertyForRent**

| propertyNo | street        | city     | postcode | type  | rooms | rent | ownerNo | staffNo | branchNo |
|------------|---------------|----------|----------|-------|-------|------|---------|---------|----------|
| PA14       | 16 Holhead    | Aberdeen | AB7 5SU  | House | 6     | 650  | CO46    | SA9     | B007     |
| PL94       | 6 Argyll St   | London   | NW2      | Flat  | 4     | 400  | CO87    | SL41    | B005     |
| PG4        | 6 Lawrence St | Glasgow  | G11 9QX  | Flat  | 3     | 350  | CO40    |         | B003     |
| PG36       | 2 Manor Rd    | Glasgow  | G32 4QX  | Flat  | 3     | 375  | CO93    | SG37    | B003     |
| PG21       | 18 Dale Rd    | Glasgow  | G12      | House | 5     | 600  | CO87    | SG37    | B003     |
| PG16       | 5 Novar Dr    | Glasgow  | G12 9AX  | Flat  | 4     | 450  | CO93    | SG14    | B003     |

# SQL JOIN

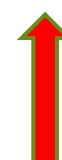
- Used to combine rows from two or more tables, based on a reference column between them.

Staff

| staffNo | fName | lName | position   | sex | DOB       | salary | branchNo |
|---------|-------|-------|------------|-----|-----------|--------|----------|
| SL21    | John  | White | Manager    | M   | 1-Oct-45  | 30000  | B005     |
| SG37    | Ann   | Beech | Assistant  | F   | 10-Nov-60 | 12000  | B003     |
| SG14    | David | Ford  | Supervisor | M   | 24-Mar-58 | 18000  | B003     |
| SA9     | Mary  | Howe  | Assistant  | F   | 19-Feb-70 | 9000   | B007     |
| SG5     | Susan | Brand | Manager    | F   | 3-Jun-40  | 24000  | B003     |
| SL41    | Julie | Lee   | Assistant  | F   | 13-Jun-65 | 9000   | B005     |

Branch

| branchNo | street       | city     | postcode |
|----------|--------------|----------|----------|
| B005     | 22 Deer Rd   | London   | SW1 4EH  |
| B007     | 16 Argyll St | Aberdeen | AB2 3SU  |
| B003     | 163 Main St  | Glasgow  | G11 9QX  |
| B004     | 32 Manse Rd  | Bristol  | BS99 1NZ |
| B002     | 56 Clover Dr | London   | NW10 6EU |



# Table Joining using Where Clause

```
SELECT table1.column, table2.column
FROM table1, table2
WHERE table1.column1 = table2.column2
```

```
SELECT Staff.staffno, Staff.fname ,Staff.branchno, Branch.city
FROM Branch, Staff
Where Branch.branchno = Staff.branchno;
```

|   | staffno | fname | branchno | city     |
|---|---------|-------|----------|----------|
| 1 | SA9     | Mary  | B007     | Aberdeen |
| 2 | SG14    | David | B003     | Glasgow  |
| 3 | SG37    | Ann   | B003     | Glasgow  |
| 4 | SG5     | Susan | B003     | Glasgow  |
| 5 | SL21    | John  | B005     | London   |
| 6 | SL41    | Julie | B005     | London   |

# Alternative to where for Joining tables

SELECT \*

FROM table1 JOIN table2.....

[ON (join\_condition)]

- ▶ SELECT Staff.staffno, Staff.fname ,Staff.branchno, Branch.city  
FROM Branch join Staff  
ON Branch.branchno = Staff.branchno;

|   | staffno | fname | branchno | city     |
|---|---------|-------|----------|----------|
| 1 | SA9     | Mary  | B007     | Aberdeen |
| 2 | SG14    | David | B003     | Glasgow  |
| 3 | SG37    | Ann   | B003     | Glasgow  |
| 4 | SG5     | Susan | B003     | Glasgow  |
| 5 | SL21    | John  | B005     | London   |
| 6 | SL41    | Julie | B005     | London   |

## ► Joining More than Two Tables

- select staff.staffno, staff.fname, propertyforrent.propertyno, privateowner.ownerno, privateowner.fname

From Staff,PropertyForRent, PrivateOwner

Where staff.staffno = propertyforrent.staffno

AND propertyforrent.ownerno = privateowner.ownerno;

|   | staffno | fname | propertyno | ownerno | fname |
|---|---------|-------|------------|---------|-------|
| 1 | SA9     | Mary  | PA14       | CO46    | Joe   |
| 2 | SG14    | David | PG16       | CO93    | Tony  |
| 3 | SG37    | Ann   | PG21       | CO87    | Carol |
| 4 | SG37    | Ann   | PG36       | CO93    | Tony  |
| 5 | SL41    | Julie | PL94       | CO87    | Carol |

## ► Joining More than Two Tables Alternative Approach

- select staff.staffno, staff.fname, propertyforrent.propertyno, privateowner.ownerno, privateowner.fname

From Staff Join PropertyForRent

ON staff.staffno = propertyforrent.staffno

Join PrivateOwner

ON propertyforrent.ownerno = privateowner.ownerno;

|   | staffno | fname | propertyno | ownerno | fname |
|---|---------|-------|------------|---------|-------|
| 1 | SA9     | Mary  | PA14       | CO46    | Joe   |
| 2 | SG14    | David | PG16       | CO93    | Tony  |
| 3 | SG37    | Ann   | PG21       | CO87    | Carol |
| 4 | SG37    | Ann   | PG36       | CO93    | Tony  |
| 5 | SL41    | Julie | PL94       | CO87    | Carol |

# Can add condition

- ▶ select staff.staffno, staff.fname, propertyforrent.propertyno, privateowner.ownerno, privateowner.fname

From Staff, ProperForRent, PrivateOwner

Where staff.staffno = propertyforrent.staffno

AND propertyforrent.ownerno = privateowner.ownerno

AND Privateowner.fname <> 'Joe'

|   | staffno | fname | propertyno | ownerno | fname |
|---|---------|-------|------------|---------|-------|
| 1 | SG14    | David | PG16       | CO93    | Tony  |
| 2 | SG37    | Ann   | PG21       | CO87    | Carol |
| 3 | SG37    | Ann   | PG36       | CO93    | Tony  |
| 4 | SL41    | Julie | PL94       | CO87    | Carol |

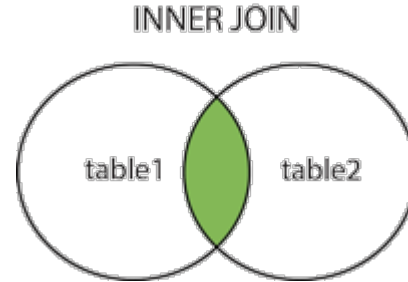
# Use of Alias with Table name

- ▶ select S.staffno, S.fname, P.propertyno, O.ownerno, O.fname  
From Staff S, ProperForRent P, PrivateOwner O  
Where S.staffno = P.staffno  
AND P.ownerno = O.ownerno  
AND O.fname <> 'Joe'

|   | staffno | fname | propertyno | ownerno | fname |
|---|---------|-------|------------|---------|-------|
| 1 | SG14    | David | PG16       | CO93    | Tony  |
| 2 | SG37    | Ann   | PG21       | CO87    | Carol |
| 3 | SG37    | Ann   | PG36       | CO93    | Tony  |
| 4 | SL41    | Julie | PL94       | CO87    | Carol |

# SQL INNER JOIN

The INNER JOIN keyword selects records that have matching values in both tables.



```
SELECT column_name(s)
FROM table1
INNER JOIN table2
ON table1.column_name = table2.column_name;
```

- ▶ `SELECT Staff.staffno, Staff.fname ,Staff.branchno, Branch.city`  
`FROM Branch inner join Staff`  
`ON Branch.branchno = Staff.branchno;`

|   | staffno | fname | branchno | city     |
|---|---------|-------|----------|----------|
| 1 | SA9     | Mary  | B007     | Aberdeen |
| 2 | SG14    | David | B003     | Glasgow  |
| 3 | SG37    | Ann   | B003     | Glasgow  |
| 4 | SG5     | Susan | B003     | Glasgow  |
| 5 | SL21    | John  | B005     | London   |
| 6 | SL41    | Julie | B005     | London   |

We need to retrieve data of staff name their numbers, their city where they are working along with their branch number.

```
SELECT Staff.staffno, Staff.fname ,Staff.branchno, Branch.city
FROM Branch
```

```
INNER JOIN Staff
```

```
ON Branch.branchno = Staff.branchno;
```

|   | staffno | fname | branchno | city     |
|---|---------|-------|----------|----------|
| 1 | SA9     | Mary  | B007     | Aberdeen |
| 2 | SG14    | David | B003     | Glasgow  |
| 3 | SG37    | Ann   | B003     | Glasgow  |
| 4 | SG5     | Susan | B003     | Glasgow  |
| 5 | SL21    | John  | B005     | London   |
| 6 | SL41    | Julie | B005     | London   |

# Join three or more tables

- ▶ select staff.staffno, staff.fname, propertyforrent.propertyno, privateowner.ownerno, privateowner.fname
- ▶ From ((Staff
- ▶ inner join propertyforrent on staff.staffno = propertyforrent.staffno)
- ▶ inner join privateowner on propertyforrent.ownerno = privateowner.ownerno);

|   | staffno | fname | propertyno | ownerno | fname |
|---|---------|-------|------------|---------|-------|
| 1 | SA9     | Mary  | PA14       | CO46    | Joe   |
| 2 | SG14    | David | PG16       | CO93    | Tony  |
| 3 | SG37    | Ann   | PG21       | CO87    | Carol |
| 4 | SG37    | Ann   | PG36       | CO93    | Tony  |
| 5 | SL41    | Julie | PL94       | CO87    | Carol |

# Previous part Review

- SQL Joins
  - Where clause
  - Join & ON
  - INNER JOIN
  - ALIAS for table name
  - Join two tables
  - Join three or more tables

**Branch**

| branchNo | street       | city     | postcode |
|----------|--------------|----------|----------|
| B005     | 22 Deer Rd   | London   | SW1 4EH  |
| B007     | 16 Argyll St | Aberdeen | AB2 3SU  |
| B003     | 163 Main St  | Glasgow  | G11 9QX  |
| B004     | 32 Manse Rd  | Bristol  | BS99 1NZ |
| B002     | 56 Clover Dr | London   | NW10 6EU |

**Staff**

| staffNo | fName | lName | position   | sex | DOB       | salary | branchNo |
|---------|-------|-------|------------|-----|-----------|--------|----------|
| SL21    | John  | White | Manager    | M   | 1-Oct-45  | 30000  | B005     |
| SG37    | Ann   | Beech | Assistant  | F   | 10-Nov-60 | 12000  | B003     |
| SG14    | David | Ford  | Supervisor | M   | 24-Mar-58 | 18000  | B003     |
| SA9     | Mary  | Howe  | Assistant  | F   | 19-Feb-70 | 9000   | B007     |
| SG5     | Susan | Brand | Manager    | F   | 3-Jun-40  | 24000  | B003     |
| SL41    | Julie | Lee   | Assistant  | F   | 13-Jun-65 | 9000   | B005     |

**PrivateOwner**

| ownerNo | fName | lName  | address                       | telNo         | eMail             |
|---------|-------|--------|-------------------------------|---------------|-------------------|
| CO46    | Joe   | Keogh  | 2 Fergus Dr, Aberdeen AB2 7SX | 01224-861212  | jkeogh@lhh.com    |
| CO87    | Carol | Farrel | 6 Achray St, Glasgow G32 9DX  | 0141-357-7419 | cfarrel@gmail.com |
| CO40    | Tina  | Murphy | 63 Well St, Glasgow G42       | 0141-943-1728 | tinam@hotmail.com |
| CO93    | Tony  | Shaw   | 12 Park Pl, Glasgow G4 0QR    | 0141-225-7025 | tony.shaw@ark.com |

**PropertyForRent**

| propertyNo | street        | city     | postcode | type  | rooms | rent | ownerNo | staffNo | branchNo |
|------------|---------------|----------|----------|-------|-------|------|---------|---------|----------|
| PA14       | 16 Holhead    | Aberdeen | AB7 5SU  | House | 6     | 650  | CO46    | SA9     | B007     |
| PL94       | 6 Argyll St   | London   | NW2      | Flat  | 4     | 400  | CO87    | SL41    | B005     |
| PG4        | 6 Lawrence St | Glasgow  | G11 9QX  | Flat  | 3     | 350  | CO40    |         | B003     |
| PG36       | 2 Manor Rd    | Glasgow  | G32 4QX  | Flat  | 3     | 375  | CO93    | SG37    | B003     |
| PG21       | 18 Dale Rd    | Glasgow  | G12      | House | 5     | 600  | CO87    | SG37    | B003     |
| PG16       | 5 Novar Dr    | Glasgow  | G12 9AX  | Flat  | 4     | 450  | CO93    | SG14    | B003     |

# Agenda

## ➤ SQL Joins

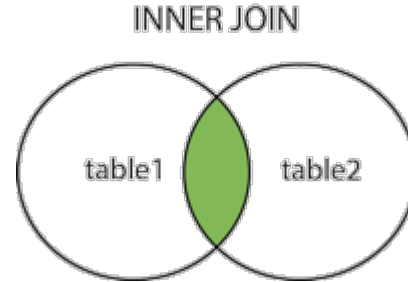
### ▶ Outer Joins

- ▶ Left Outer Join
- ▶ Right Outer Join
- ▶ Full Outer Join

### ▶ Self Join

# SQL INNER JOIN

The INNER JOIN keyword selects records that have matching values in both tables.



```
SELECT column_name(s)
FROM table1
INNER JOIN table2
ON table1.column_name = table2.column_name;
```

► select \* from branch inner join staff on branch.branchno = staff.branchno

| branchNo | street       | city     | postCode | staffNo | fName | lName | position   | sex | DOB        | salary | branchNo |
|----------|--------------|----------|----------|---------|-------|-------|------------|-----|------------|--------|----------|
| B007     | 16 Argyll St | Aberdeen | AB2 3SU  | SA9     | Mary  | Howe  | Assistant  | F   | 1970-02-19 | 9000   | B007     |
| B003     | 163 Main St  | Glasgow  | G11 9QX  | SG14    | David | Ford  | Supervisor | M   | 1958-11-24 | 18000  | B003     |
| B003     | 163 Main St  | Glasgow  | G11 9QX  | SG37    | Ann   | Beech | Assistant  | F   | 1960-10-11 | 12000  | B003     |
| B003     | 163 Main St  | Glasgow  | G11 9QX  | SG5     | Susan | Brand | Manager    | F   | 1940-06-03 | 24000  | B003     |
| B005     | 22 Deer Rd   | London   | SW1 4EH  | SL21    | John  | White | Manager    | M   | 1945-10-01 | 30000  | B005     |
| B005     | 22 Deer Rd   | London   | SW1 4EH  | SL41    | Julie | Lee   | Assistant  | F   | 1965-06-13 | 9000   | B005     |

► select \* from staff inner join branch on branch.branchno = staff.branchno

| staffNo | fName | lName | position   | sex | DOB        | salary | branchNo | branchNo | street       | city     | postCode |
|---------|-------|-------|------------|-----|------------|--------|----------|----------|--------------|----------|----------|
| SA9     | Mary  | Howe  | Assistant  | F   | 1970-02-19 | 9000   | B007     | B007     | 16 Argyll St | Aberdeen | AB2 3SU  |
| SG14    | David | Ford  | Supervisor | M   | 1958-11-24 | 18000  | B003     | B003     | 163 Main St  | Glasgow  | G11 9QX  |
| SG37    | Ann   | Beech | Assistant  | F   | 1960-10-11 | 12000  | B003     | B003     | 163 Main St  | Glasgow  | G11 9QX  |
| SG5     | Susan | Brand | Manager    | F   | 1940-06-03 | 24000  | B003     | B003     | 163 Main St  | Glasgow  | G11 9QX  |
| SL21    | John  | White | Manager    | M   | 1945-10-01 | 30000  | B005     | B005     | 22 Deer Rd   | London   | SW1 4EH  |
| SL41    | Julie | Lee   | Assistant  | F   | 1965-06-13 | 9000   | B005     | B005     | 22 Deer Rd   | London   | SW1 4EH  |

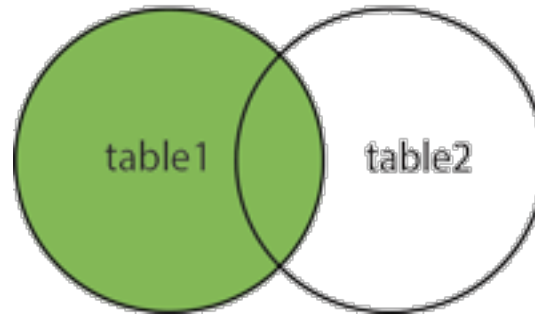
# Outer Joins

- ▶ Left Outer Join
- ▶ Right Outer Join
- ▶ Full Outer Join

# SQL LEFT OUTER JOIN

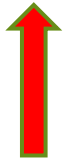
- The LEFT JOIN keyword returns all records from the left table (table1), and the matched records from the right table (table2). The result is NULL from the right side, if there is no match.

LEFT JOIN



Branch

| branchNo | street       | city     | postcode |
|----------|--------------|----------|----------|
| B005     | 22 Deer Rd   | London   | SW1 4EH  |
| B007     | 16 Argyll St | Aberdeen | AB2 3SU  |
| B003     | 163 Main St  | Glasgow  | G11 9QX  |
| B004     | 32 Manse Rd  | Bristol  | BS99 1NZ |
| B002     | 56 Clover Dr | London   | NW10 6EU |



Staff

| staffNo | fName | lName | position   | sex | DOB       | salary | branchNo |
|---------|-------|-------|------------|-----|-----------|--------|----------|
| SL21    | John  | White | Manager    | M   | 1-Oct-45  | 30000  | B005     |
| SG37    | Ann   | Beech | Assistant  | F   | 10-Nov-60 | 12000  | B003     |
| SG14    | David | Ford  | Supervisor | M   | 24-Mar-58 | 18000  | B003     |
| SA9     | Mary  | Howe  | Assistant  | F   | 19-Feb-70 | 9000   | B007     |
| SG5     | Susan | Brand | Manager    | F   | 3-Jun-40  | 24000  | B003     |
| SL41    | Julie | Lee   | Assistant  | F   | 13-Jun-65 | 9000   | B005     |



```
SELECT column_name(s)
FROM table1
LEFT JOIN table2
ON table1.column_name = table2.column_name;
```

```
SELECT Branch.branchno, Branch.city, staff.staffno, staff.fname
FROM Branch
LEFT JOIN Staff ON Branch.branchno = staff.branchno
ORDER BY branch.branchno;
```

| branchno | city     | staffno | fname |
|----------|----------|---------|-------|
| B002     | London   | NULL    | NULL  |
| B003     | Glasgow  | SG14    | David |
| B003     | Glasgow  | SG37    | Ann   |
| B003     | Glasgow  | SG5     | Susan |
| B004     | Bristol  | NULL    | NULL  |
| B005     | London   | SL21    | John  |
| B005     | London   | SL41    | Julie |
| B007     | Aberdeen | SA9     | Mary  |

# Generate a report for total salary of each branch

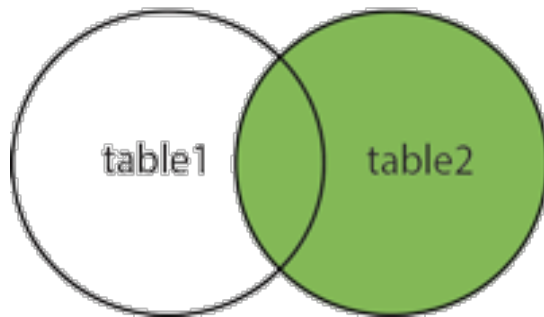
- ▶ `select branch.branchno, branch.city, sum(staff.salary) As [Branch Salary]`  
`from branch left join staff on Branch.branchno = staff.branchno`  
`Group by branch.branchNo,branch.city`  
`order by branchno`

| branchno | city     | Branch Salary |
|----------|----------|---------------|
| B002     | London   | NULL          |
| B003     | Glasgow  | 54000         |
| B004     | Bristol  | NULL          |
| B005     | London   | 39000         |
| B007     | Aberdeen | 9000          |

# SQL RIGHT OUTER JOIN

- The RIGHT JOIN keyword returns all records from the right table (table2), and the matched records from the left table (table1). The result is NULL from the left side, when there is no match.

RIGHT JOIN



Staff

| staffNo | fName | lName | position   | sex | DOB       | salary | branchNo |
|---------|-------|-------|------------|-----|-----------|--------|----------|
| SL21    | John  | White | Manager    | M   | 1-Oct-45  | 30000  | B005     |
| SG37    | Ann   | Beech | Assistant  | F   | 10-Nov-60 | 12000  | B003     |
| SG14    | David | Ford  | Supervisor | M   | 24-Mar-58 | 18000  | B003     |
| SA9     | Mary  | Howe  | Assistant  | F   | 19-Feb-70 | 9000   | B007     |
| SG5     | Susan | Brand | Manager    | F   | 3-Jun-40  | 24000  | B003     |
| SL41    | Julie | Lee   | Assistant  | F   | 13-Jun-65 | 9000   | B005     |

PropertyForRent

| propertyNo | street        | city     | postcode | type  | rooms | rent | ownerNo | staffNo | branchNo |
|------------|---------------|----------|----------|-------|-------|------|---------|---------|----------|
| PA14       | 16 Holhead    | Aberdeen | AB7 5SU  | House | 6     | 650  | CO46    | SA9     | B007     |
| PL94       | 6 Argyll St   | London   | NW2      | Flat  | 4     | 400  | CO87    | SL41    | B005     |
| PG4        | 6 Lawrence St | Glasgow  | G11 9QX  | Flat  | 3     | 350  | CO40    |         | B003     |
| PG36       | 2 Manor Rd    | Glasgow  | G32 4QX  | Flat  | 3     | 375  | CO93    | SG37    | B003     |
| PG21       | 18 Dale Rd    | Glasgow  | G12      | House | 5     | 600  | CO87    | SG37    | B003     |
| PG16       | 5 Novar Dr    | Glasgow  | G12 9AX  | Flat  | 4     | 450  | CO93    | SG14    | B003     |

- ▶ `SELECT column_name(s)`  
`FROM table1`  
`RIGHT JOIN table2`  
`ON table1.column_name = table2.column_name;`
- ▶ `select * from staff right join PropertyForRent on staff.staffno = PropertyForRent.staffno`

| staffNo | fName | lName | position   | sex  | DOB        | salary | branchNo | propertyNo | street        | city     | postcode | TYPE  | rooms | rent | ownerNo | staffNo | branchNo |
|---------|-------|-------|------------|------|------------|--------|----------|------------|---------------|----------|----------|-------|-------|------|---------|---------|----------|
| SA9     | Mary  | Howe  | Assistant  | F    | 1970-02-19 | 9000   | B007     | PA14       | 16 Holhead    | Aberdeen | AB7 5SU  | House | 6     | 650  | CO46    | SA9     | B007     |
| SG14    | David | Ford  | Supervisor | M    | 1958-11-24 | 18000  | B003     | PG16       | 5 Novar Dr    | Glasgow  | G12 9AX  | Flat  | 4     | 450  | CO93    | SG14    | B003     |
| SG37    | Ann   | Beech | Assistant  | F    | 1960-10-11 | 12000  | B003     | PG21       | 18 Dale Rd    | Glasgow  | G12      | House | 5     | 600  | CO87    | SG37    | B003     |
| SG37    | Ann   | Beech | Assistant  | F    | 1960-10-11 | 12000  | B003     | PG36       | 2 Manor Rd    | Glasgow  | G32 4QX  | Flat  | 3     | 375  | CO93    | SG37    | B003     |
| NULL    | NULL  | NULL  | NULL       | NULL | NULL       | NULL   | NULL     | PG4        | 6 Lawrence St | Glasgow  | G11 9QX  | Flat  | 3     | 350  | CO40    | NULL    | B003     |
| SL41    | Julie | Lee   | Assistant  | F    | 1965-06-13 | 9000   | B005     | PL94       | 6 Argyll St   | London   | NW2      | Flat  | 4     | 400  | CO87    | SL41    | B005     |

SG5  
SL21

# Detail of all properties with the detail of staff who assigned each property

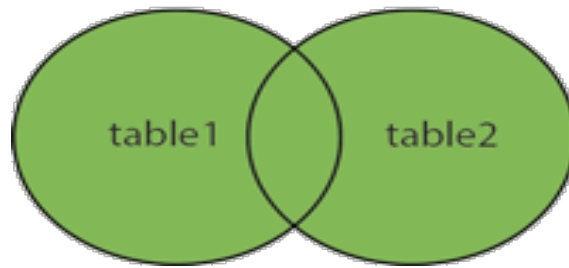
- ▶ `select staff.staffno, staff.fname,staff.position, propertyforrent.*  
from staff right join PropertyForRent on staff.staffno =PropertyForRent.staffno  
order by propertyforrent.propertyNo`

| staffno | fname | position   | propertyNo | street        | city     | postcode | TYPE  | rooms | rent | ownerNo | staffNo | branchNo |
|---------|-------|------------|------------|---------------|----------|----------|-------|-------|------|---------|---------|----------|
| SA9     | Mary  | Assistant  | PA14       | 16 Holhead    | Aberdeen | AB7 5SU  | House | 6     | 650  | CO46    | SA9     | B007     |
| SG14    | David | Supervisor | PG16       | 5 Novar Dr    | Glasgow  | G12 9AX  | Flat  | 4     | 450  | CO93    | SG14    | B003     |
| SG37    | Ann   | Assistant  | PG21       | 18 Dale Rd    | Glasgow  | G12      | House | 5     | 600  | CO87    | SG37    | B003     |
| SG37    | Ann   | Assistant  | PG36       | 2 Manor Rd    | Glasgow  | G32 4QX  | Flat  | 3     | 375  | CO93    | SG37    | B003     |
| NULL    | NULL  | NULL       | PG4        | 6 Lawrence St | Glasgow  | G11 9QX  | Flat  | 3     | 350  | CO40    | NULL    | B003     |
| SL41    | Julie | Assistant  | PL94       | 6 Argyll St   | London   | NW2      | Flat  | 4     | 400  | CO87    | SL41    | B005     |

# SQL FULL OUTER JOIN

- ▶ The FULL OUTER JOIN keyword returns all records when there is a match in left (table1) or right (table2) table records.
- ▶ **Note:** FULL OUTER JOIN can potentially return very large result-sets!
- ▶ **Tip:** FULL OUTER JOIN and FULL JOIN are the same.

FULL OUTER JOIN



## Staff

| staffNo | fName | IName | position   | sex | DOB       | salary | branchNo |
|---------|-------|-------|------------|-----|-----------|--------|----------|
| SL21    | John  | White | Manager    | M   | 1-Oct-45  | 30000  | B005     |
| SG37    | Ann   | Beech | Assistant  | F   | 10-Nov-60 | 12000  | B003     |
| SG14    | David | Ford  | Supervisor | M   | 24-Mar-58 | 18000  | B003     |
| SA9     | Mary  | Howe  | Assistant  | F   | 19-Feb-70 | 9000   | B007     |
| SG5     | Susan | Brand | Manager    | F   | 3-Jun-40  | 24000  | B003     |
| SL41    | Julie | Lee   | Assistant  | F   | 13-Jun-65 | 9000   | B005     |

## PropertyForRent

| propertyNo | street        | city     | postcode | type  | rooms | rent | ownerNo | staffNo | branchNo |
|------------|---------------|----------|----------|-------|-------|------|---------|---------|----------|
| PA14       | 16 Holhead    | Aberdeen | AB7 5SU  | House | 6     | 650  | CO46    | SA9     | B007     |
| PL94       | 6 Argyll St   | London   | NW2      | Flat  | 4     | 400  | CO87    | SL41    | B005     |
| PG4        | 6 Lawrence St | Glasgow  | G11 9QX  | Flat  | 3     | 350  | CO40    |         | B003     |
| PG36       | 2 Manor Rd    | Glasgow  | G32 4QX  | Flat  | 3     | 375  | CO93    | SG37    | B003     |
| PG21       | 18 Dale Rd    | Glasgow  | G12      | House | 5     | 600  | CO87    | SG37    | B003     |
| PG16       | 5 Novar Dr    | Glasgow  | G12 9AX  | Flat  | 4     | 450  | CO93    | SG14    | B003     |

```

SELECT column_name(s)
FROM table1
FULL OUTER JOIN table2
ON table1.column_name = table2.column_name
WHERE condition;

```

```

select * from staff full join PropertyForRent on staff.staffno = PropertyForRent.staffno

```

| staffNo | fName | lName | position   | sex  | DOB        | salary | branchNo | propertyNo | street        | city     | postcode | TYPE  | rooms | rent | ownerNo | staffNo | branchNo |
|---------|-------|-------|------------|------|------------|--------|----------|------------|---------------|----------|----------|-------|-------|------|---------|---------|----------|
| SA9     | Mary  | Howe  | Assistant  | F    | 1970-02-19 | 9000   | B007     | PA14       | 16 Holhead    | Aberdeen | AB7 5SU  | House | 6     | 650  | CO46    | SA9     | B007     |
| SG14    | David | Ford  | Supervisor | M    | 1958-11-24 | 18000  | B003     | PG16       | 5 Novar Dr    | Glasgow  | G12 9AX  | Flat  | 4     | 450  | CO93    | SG14    | B003     |
| SG37    | Ann   | Beech | Assistant  | F    | 1960-10-11 | 12000  | B003     | PG21       | 18 Dale Rd    | Glasgow  | G12      | House | 5     | 600  | CO87    | SG37    | B003     |
| SG37    | Ann   | Beech | Assistant  | F    | 1960-10-11 | 12000  | B003     | PG36       | 2 Manor Rd    | Glasgow  | G32 4QX  | Flat  | 3     | 375  | CO93    | SG37    | B003     |
| SG5     | Susan | Brand | Manager    | F    | 1940-06-03 | 24000  | B003     | NULL       | NULL          | NULL     | NULL     | NULL  | NULL  | NULL | NULL    | NULL    | NULL     |
| SL21    | John  | White | Manager    | M    | 1945-10-01 | 30000  | B005     | NULL       | NULL          | NULL     | NULL     | NULL  | NULL  | NULL | NULL    | NULL    | NULL     |
| SL41    | Julie | Lee   | Assistant  | F    | 1965-06-13 | 9000   | B005     | PL94       | 6 Argyll St   | London   | NW2      | Flat  | 4     | 400  | CO87    | SL41    | B005     |
| NULL    | NULL  | NULL  | NULL       | NULL | NULL       | NULL   | NULL     | PG4        | 6 Lawrence St | Glasgow  | G11 9QX  | Flat  | 3     | 350  | CO40    | NULL    | B003     |

# Is there any staff number who did not assign a property and there is any property which is not assigned to any staff number at branch B003

- ▶ `select staff.staffno, staff.fname, propertyforrent.propertyNo, PropertyForRent.branchno  
from staff full join PropertyForRent on staff.staffno = PropertyForRent.staffno  
where propertyforrent.branchno='B003' or staff.branchno ='B003'`

| staffno | fname | propertyNo | branchno |
|---------|-------|------------|----------|
| SG14    | David | PG16       | B003     |
| SG37    | Ann   | PG21       | B003     |
| SG37    | Ann   | PG36       | B003     |
| SG5     | Susan | NULL       | NULL     |
| NULL    | NULL  | PG4        | B003     |

- ▶ select staff.staffno, staff.fname, propertyforrent.propertyNo, PropertyForRent.branchno  
from staff full join PropertyForRent on staff.staffno = PropertyForRent.staffno  
where propertyforrent.branchno='B003'

| staffno | fname | propertyNo | branchno |
|---------|-------|------------|----------|
| SG14    | David | PG16       | B003     |
| SG37    | Ann   | PG21       | B003     |
| SG37    | Ann   | PG36       | B003     |
| NULL    | NULL  | PG4        | B003     |

# Self Join

- ▶ Used when we want to join a table with the same table
- ▶ `SELECT column_name(s)`  
`FROM table1 T1, table1 T2`  
`WHERE condition;`

**Staff**

| staffNo | fName | lName | position   | sex | DOB       | salary | branchNo |
|---------|-------|-------|------------|-----|-----------|--------|----------|
| SL21    | John  | White | Manager    | M   | 1-Oct-45  | 30000  | B005     |
| SG37    | Ann   | Beech | Assistant  | F   | 10-Nov-60 | 12000  | B003     |
| SG14    | David | Ford  | Supervisor | M   | 24-Mar-58 | 18000  | B003     |
| SA9     | Mary  | Howe  | Assistant  | F   | 19-Feb-70 | 9000   | B007     |
| SG5     | Susan | Brand | Manager    | F   | 3-Jun-40  | 24000  | B003     |
| SL41    | Julie | Lee   | Assistant  | F   | 13-Jun-65 | 9000   | B005     |

► We want to pair Managers and Assistants as teams, so need data to tie them up

► SELECT s1.staffno, s1.position, s2.staffno,s2.position, s1.branchno

► FROM staff s1, staff s2

► WHERE s1.position != s2.position

AND s1.position = 'manager'      AND s2.position = 'assistant'

order by s1.branchno

| staffno | position | staffno | position  |
|---------|----------|---------|-----------|
| SG5     | Manager  | SA9     | Assistant |
| SG5     | Manager  | SG37    | Assistant |
| SG5     | Manager  | SL41    | Assistant |
| SL21    | Manager  | SA9     | Assistant |
| SL21    | Manager  | SG37    | Assistant |
| SL21    | Manager  | SL41    | Assistant |



# COMSATS University Islamabad, Lahore Campus

## Department of Computer Science

### Lab Assignment

|                    |                       |                |        |               |        |
|--------------------|-----------------------|----------------|--------|---------------|--------|
| Course Title:      | Database Systems I    | Course Code:   | CSC371 | Credit Hours: | 4(3,1) |
| Course Instructor: | Mahrukh Saleem        | Program Name:  | BS CS  |               |        |
| Semester:          | 4 <sup>th</sup>       | Batch:         | FA24   | Section:      | B,C    |
| Date:              | 26-03-2026            |                |        |               |        |
| Time Allowed:      | 1 Hour and 30 Minutes | Maximum Marks: | 45     |               |        |
| Student's Name:    |                       | Reg. No.       |        |               |        |

#### Important Instructions / Guidelines:

- Plan and manage your time accordingly.
- You are required to submit the word file and the .sql file on google classroom.
- Name your file after your registration number. For example: FA24-BCS-000

#### Question 1:

[Marks: 15]

Create the following tables with reasonable constraints and insert given values into relevant tables

| Paintings |                    |           |              |
|-----------|--------------------|-----------|--------------|
| id        | name               | artist_id | listed_price |
| 11        | Miracle            | 1         | 300.00       |
| 12        | Sunshine           | 1         | 700.00       |
| 13        | Pretty woman       | 2         | 2800.00      |
| 14        | Handsome man       | 2         | 2300.00      |
| 15        | Barbie             | 3         | 250.00       |
| 16        | Cool painting      | 3         | 5000.00      |
| 17        | Black square #1000 | 3         | 50.00        |
| 18        | Mountains          | 4         | 1300.00      |

| Artists |            |           |
|---------|------------|-----------|
| id      | first_name | last_name |
| 1       | Thomas     | Black     |
| 2       | Kate       | Smith     |
| 3       | Natali     | Wein      |
| 4       | Francesco  | Benelli   |

| Collectors |            |           |
|------------|------------|-----------|
| id         | first_name | last_name |
| 101        | Brandon    | Cooper    |
| 102        | Laura      | Fisher    |
| 103        | Christina  | Buffet    |
| 104        | Steve      | Stevenson |

| Sales |            |             |           |              |             |
|-------|------------|-------------|-----------|--------------|-------------|
| id    | date       | painting_id | artist_id | collector_id | sales_price |
| 1001  | 2021-11-01 | 13          | 2         | 104          | 2500.00     |
| 1002  | 2021-11-10 | 14          | 2         | 102          | 2300.00     |
| 1003  | 2021-11-10 | 11          | 1         | 102          | 300.00      |
| 1004  | 2021-11-15 | 16          | 3         | 103          | 4000.00     |
| 1005  | 2021-11-22 | 15          | 3         | 103          | 200.00      |
| 1006  | 2021-11-22 | 17          | 3         | 103          | 50.00       |



# COMSATS University Islamabad, Lahore Campus

## Department of Computer Science

---

### Question 2:

[Marks: 30]

- 1) Display the number of sold paintings of each artist. [3 marks]
- 2) Show the number of paintings sold on “2021-11-10”. [3 marks]
- 3) Display the names of paintings and their listed prices whose names ends at “e”. [3 marks]
- 4) Display the sum of listed price of paintings of each artist. [3 marks]
- 5) Update the listed price of those paintings whose name begins with B or M, to 9000. [3 marks]
- 6) Display the full name of Artists as a single column. [3 marks]
- 7) Display the average sales price of Artist id 3 with his name [3 marks]
- 8) Remove collector id 103 from the records. [3 marks]
- 9) List the painting id whose sales price is greater than the average sales price of all paintings. [3 marks]
- 10) Show the total number of paintings in the database. [3 marks]

*Best of luck.*

# CSC270-Database Systems I (Lab 5) (Spring 2026)

Mahrukh Saleem

[mahrukhsaleem@cuilahore.edu.pk](mailto:mahrukhsaleem@cuilahore.edu.pk)

**Branch**

| branchNo | street       | city     | postcode |
|----------|--------------|----------|----------|
| B005     | 22 Deer Rd   | London   | SW1 4EH  |
| B007     | 16 Argyll St | Aberdeen | AB2 3SU  |
| B003     | 163 Main St  | Glasgow  | G11 9QX  |
| B004     | 32 Manse Rd  | Bristol  | BS99 1NZ |
| B002     | 56 Clover Dr | London   | NW10 6EU |

**Staff**

| staffNo | fName | lName | position   | sex | DOB       | salary | branchNo |
|---------|-------|-------|------------|-----|-----------|--------|----------|
| SL21    | John  | White | Manager    | M   | 1-Oct-45  | 30000  | B005     |
| SG37    | Ann   | Beech | Assistant  | F   | 10-Nov-60 | 12000  | B003     |
| SG14    | David | Ford  | Supervisor | M   | 24-Mar-58 | 18000  | B003     |
| SA9     | Mary  | Howe  | Assistant  | F   | 19-Feb-70 | 9000   | B007     |
| SG5     | Susan | Brand | Manager    | F   | 3-Jun-40  | 24000  | B003     |
| SL41    | Julie | Lee   | Assistant  | F   | 13-Jun-65 | 9000   | B005     |

**PrivateOwner**

| ownerNo | fName | lName  | address                       | telNo         | eMail             |
|---------|-------|--------|-------------------------------|---------------|-------------------|
| CO46    | Joe   | Keogh  | 2 Fergus Dr, Aberdeen AB2 7SX | 01224-861212  | jkeogh@lhh.com    |
| CO87    | Carol | Farrel | 6 Achray St, Glasgow G32 9DX  | 0141-357-7419 | cfarrel@gmail.com |
| CO40    | Tina  | Murphy | 63 Well St, Glasgow G42       | 0141-943-1728 | tinam@hotmail.com |
| CO93    | Tony  | Shaw   | 12 Park Pl, Glasgow G4 0QR    | 0141-225-7025 | tony.shaw@ark.com |

**PropertyForRent**

| propertyNo | street        | city     | postcode | type  | rooms | rent | ownerNo | staffNo | branchNo |
|------------|---------------|----------|----------|-------|-------|------|---------|---------|----------|
| PA14       | 16 Holhead    | Aberdeen | AB7 5SU  | House | 6     | 650  | CO46    | SA9     | B007     |
| PL94       | 6 Argyll St   | London   | NW2      | Flat  | 4     | 400  | CO87    | SL41    | B005     |
| PG4        | 6 Lawrence St | Glasgow  | G11 9QX  | Flat  | 3     | 350  | CO40    |         | B003     |
| PG36       | 2 Manor Rd    | Glasgow  | G32 4QX  | Flat  | 3     | 375  | CO93    | SG37    | B003     |
| PG21       | 18 Dale Rd    | Glasgow  | G12      | House | 5     | 600  | CO87    | SG37    | B003     |
| PG16       | 5 Novar Dr    | Glasgow  | G12 9AX  | Flat  | 4     | 450  | CO93    | SG14    | B003     |

# SQL Sub Query

- ▶ A subquery is an SQL query nested inside a larger query.
- ▶ A subquery may occur in :
  - ▶ A SELECT clause
  - ▶ A FROM clause
  - ▶ A WHERE clause
- ▶ A subquery can be treated as an inner query, which is a SQL query placed as a part of another query called as outer query.
- ▶ The inner query executes first before its parent query so that the results of inner query can be passed to the outer query.
- ▶ Must be enclosed in parentheses

- ▶ Usually added within the WHERE Clause of another SQL SELECT statement
- ▶ You can use the comparison operators, such as >, <, or =.
- ▶ The comparison operator can also be a multiple-row operator, such as IN, ANY, or ALL.
- ▶ You can use a subquery in a SELECT, INSERT, DELETE, or UPDATE statement to perform the following tasks:
  - ▶ Compare an expression to the result of the query.
  - ▶ Determine if an expression is included in the results of the query.
  - ▶ Check whether the query selects any rows.

# Staff members detail working at street '163 Main St'

- ▶ Select staff.\* from staff,branch Where staff.branchno = branch.branchno and branch.street = "163 Main St"
- ▶ Execute From clause and fetch both tables data.
- ▶ Execute where clause and Create a temporary table with matching records
- ▶ Display

| staffNo | fName | lName | position   | sex | DOB        | salary | branchNo |
|---------|-------|-------|------------|-----|------------|--------|----------|
| SG14    | David | Ford  | Supervisor | M   | 1958-11-24 | 18000  | B003     |
| SG37    | Ann   | Beech | Assistant  | F   | 1960-10-11 | 12000  | B003     |
| SG5     | Susan | Brand | Manager    | F   | 1940-06-03 | 24000  | B003     |

Branch

| branchNo | street       | city     | postcode |
|----------|--------------|----------|----------|
| B005     | 22 Deer Rd   | London   | SW1 4EH  |
| B007     | 16 Argyll St | Aberdeen | AB2 3SU  |
| B003     | 163 Main St  | Glasgow  | G11 9QX  |
| B004     | 32 Manse Rd  | Bristol  | BS99 1NZ |
| B002     | 56 Clover Dr | London   | NW10 6EU |

Staff

| staffNo | fName | lName | position   | sex | DOB       | salary | branchNo |
|---------|-------|-------|------------|-----|-----------|--------|----------|
| SL21    | John  | White | Manager    | M   | 1-Oct-45  | 30000  | B005     |
| SG37    | Ann   | Beech | Assistant  | F   | 10-Nov-60 | 12000  | B003     |
| SG14    | David | Ford  | Supervisor | M   | 24-Mar-58 | 18000  | B003     |
| SA9     | Mary  | Howe  | Assistant  | F   | 19-Feb-70 | 9000   | B007     |
| SG5     | Susan | Brand | Manager    | F   | 3-Jun-40  | 24000  | B003     |
| SL41    | Julie | Lee   | Assistant  | F   | 13-Jun-65 | 9000   | B005     |

# Alternate way with subquery

- ▶ Seelct staff.\* from staff where branchno=(select branchno from branch where street = '163 Main st')
- ▶ While we just need one record from branch table and matching of that record from staff table
- ▶ Only three records

| staffNo | fName | lName | position   | sex | DOB        | salary | branchNo |
|---------|-------|-------|------------|-----|------------|--------|----------|
| SG14    | David | Ford  | Supervisor | M   | 1958-11-24 | 18000  | B003     |
| SG37    | Ann   | Beech | Assistant  | F   | 1960-10-11 | 12000  | B003     |
| SG5     | Susan | Brand | Manager    | F   | 1940-06-03 | 24000  | B003     |

# Can you find out branch number and its city of those staff members who are drawing more than 10000 salary.

- ▶ `select distinct branch.branchno, branch.city from branch  
join staff on branch.branchno = staff.branchno and salary > 10000`
- ▶ `select branch.branchno, branch.city from branch  
where branch.branchno in (select branchno from staff where salary > 10000)`

| branchno | city    |
|----------|---------|
| B003     | Glasgow |
| B005     | London  |

| branchno | city    |
|----------|---------|
| B003     | Glasgow |
| B005     | London  |

# Subqueries with INSERT statement

## ► MySQL

► INSERT INTO staff2 SELECT \* FROM staff WHERE position = 'assistant' OR position= 'supervisor'

## ► SQL Server

► SELECT \* INTO staff2 FROM staff WHERE position = 'assistant' or position = 'supervisor'

► Select \* from staff2

| staffNo | fName | lName | position   | sex | DOB        | salary | branchNo |
|---------|-------|-------|------------|-----|------------|--------|----------|
| SA9     | Mary  | Howe  | Assistant  | F   | 1970-02-19 | 9000   | B007     |
| SG14    | David | Ford  | Supervisor | M   | 1958-11-24 | 18000  | B003     |
| SG37    | Ann   | Beech | Assistant  | F   | 1960-10-11 | 12000  | B003     |
| SL41    | Julie | Lee   | Assistant  | F   | 1965-06-13 | 9000   | B005     |

# Subqueries with UPDATE statement

- update staff2 set salary= salary+5000
- where salary < (select avg(salary) from staff2)

| staffNo | fName | lName | position   | sex | DOB        | salary | branchNo |
|---------|-------|-------|------------|-----|------------|--------|----------|
| SA9     | Mary  | Howe  | Assistant  | F   | 1970-02-19 | 14000  | B007     |
| SG14    | David | Ford  | Supervisor | M   | 1958-11-24 | 18000  | B003     |
| SG37    | Ann   | Beech | Assistant  | F   | 1960-10-11 | 12000  | B003     |
| SL41    | Julie | Lee   | Assistant  | F   | 1965-06-13 | 14000  | B005     |

# Subqueries with DELETE statement

- Delete from staff2 where branchno in (select branchno from branch where city = 'London')

| staffNo | fName | lName | position   | sex | DOB        | salary | branchNo |
|---------|-------|-------|------------|-----|------------|--------|----------|
| SA9     | Mary  | Howe  | Assistant  | F   | 1970-02-19 | 14000  | B007     |
| SG14    | David | Ford  | Supervisor | M   | 1958-11-24 | 18000  | B003     |
| SG37    | Ann   | Beech | Assistant  | F   | 1960-10-11 | 12000  | B003     |

# Single Row Subqueries

- ▶ Returns zero or one row to the outer SQL statement
- ▶ Select staff.\* from staff where branchno=(select branchno from branch where street ='163 Main st')

| staffNo | fName | lName | position   | sex | DOB        | salary | branchNo |
|---------|-------|-------|------------|-----|------------|--------|----------|
| SG14    | David | Ford  | Supervisor | M   | 1958-11-24 | 18000  | B003     |
| SG37    | Ann   | Beech | Assistant  | F   | 1960-10-11 | 12000  | B003     |
| SG5     | Susan | Brand | Manager    | F   | 1940-06-03 | 24000  | B003     |

# Multiple Row Subqueries

- ▶ `select branch.branchno, branch.city from branch  
where branch.branchno in (select branchno from staff where salary > 10000)`

| branchno | city    |
|----------|---------|
| B003     | Glasgow |
| B005     | London  |

# SQL Nested subqueries

- ▶ A subquery can be nested inside other subqueries
- ▶ *List the properties that are handled by staff who work in the branch at '163 Main St'.*
- ▶ **SELECT** propertyNo, street, city, postcode, type, rooms, rent
- ▶ **FROM** PropertyForRent
- ▶ **WHERE** staffNo IN (**SELECT** staffNo **FROM** Staff
- ▶ **WHERE** branchNo = (**SELECT** branchNo **FROM** Branch
- ▶ **WHERE** street = '163 Main St'));

| propertyNo | street     | city    | postcode | type  | rooms | rent |
|------------|------------|---------|----------|-------|-------|------|
| PG16       | 5 Novar Dr | Glasgow | G12 9AX  | Flat  | 4     | 450  |
| PG21       | 18 Dale Rd | Glasgow | G12      | House | 5     | 600  |
| PG36       | 2 Manor Rd | Glasgow | G32 4QX  | Flat  | 3     | 375  |

# Use of ANY/SOME and ALL

- ▶ The keywords ANY and ALL may be used with subqueries that produce a single column of numbers
- ▶ *Find all staff whose salary is larger than the salary of at least one member of staff at branch B003.*
- ▶ **SELECT** staffNo, fName, lName, position, salary  
**FROM** Staff **WHERE** salary > **SOME** (**SELECT** salary **FROM** Staff **WHERE** branchNo = 'B003');

| staffNo | fName | lName | position   | salary |
|---------|-------|-------|------------|--------|
| SG14    | David | Ford  | Supervisor | 18000  |
| SG5     | Susan | Brand | Manager    | 24000  |
| SL21    | John  | White | Manager    | 30000  |

# Use of ALL

- *Find all staff whose salary is larger than the salary of every member of staff at branch B003.*
- **SELECT** staffNo, fName, lName, position, salary **FROM** Staff
- **WHERE** salary > **ALL** (**SELECT** salary **FROM** Staff
- **WHERE** branchNo = 'B003');

| staffNo | fName | lName | position | salary |
|---------|-------|-------|----------|--------|
| SL21    | John  | White | Manager  | 30000  |

# CSC270-Database Systems

## Lab 6 (Spring 2026)

Mahrukh Saleem

[mahrukhsaleem@cuilahore.edu.pk](mailto:mahrukhsaleem@cuilahore.edu.pk)

## Branch

| branchNo | street       | city     | postcode |
|----------|--------------|----------|----------|
| B005     | 22 Deer Rd   | London   | SW1 4EH  |
| B007     | 16 Argyll St | Aberdeen | AB2 3SU  |
| B003     | 163 Main St  | Glasgow  | G11 9QX  |
| B004     | 32 Manse Rd  | Bristol  | BS99 1NZ |
| B002     | 56 Clover Dr | London   | NW10 6EU |

## Staff

| staffNo | fName | lName | position   | sex | DOB       | salary | branchNo |
|---------|-------|-------|------------|-----|-----------|--------|----------|
| SL21    | John  | White | Manager    | M   | 1-Oct-45  | 30000  | B005     |
| SG37    | Ann   | Beech | Assistant  | F   | 10-Nov-60 | 12000  | B003     |
| SG14    | David | Ford  | Supervisor | M   | 24-Mar-58 | 18000  | B003     |
| SA9     | Mary  | Howe  | Assistant  | F   | 19-Feb-70 | 9000   | B007     |
| SG5     | Susan | Brand | Manager    | F   | 3-Jun-40  | 24000  | B003     |
| SL41    | Julie | Lee   | Assistant  | F   | 13-Jun-65 | 9000   | B005     |

## PrivateOwner

| ownerNo | fName | lName  | address                       | telNo         | eMail             |
|---------|-------|--------|-------------------------------|---------------|-------------------|
| CO46    | Joe   | Keogh  | 2 Fergus Dr, Aberdeen AB2 7SX | 01224-861212  | jkeogh@lhh.com    |
| CO87    | Carol | Farrel | 6 Achray St, Glasgow G32 9DX  | 0141-357-7419 | cfarrel@gmail.com |
| CO40    | Tina  | Murphy | 63 Well St, Glasgow G42       | 0141-943-1728 | tinam@hotmail.com |
| CO93    | Tony  | Shaw   | 12 Park Pl, Glasgow G4 0QR    | 0141-225-7025 | tony.shaw@ark.com |

## PropertyForRent

| propertyNo | street        | city     | postcode | type  | rooms | rent | ownerNo | staffNo | branchNo |
|------------|---------------|----------|----------|-------|-------|------|---------|---------|----------|
| PA14       | 16 Holhead    | Aberdeen | AB7 5SU  | House | 6     | 650  | CO46    | SA9     | B007     |
| PL94       | 6 Argyll St   | London   | NW2      | Flat  | 4     | 400  | CO87    | SL41    | B005     |
| PG4        | 6 Lawrence St | Glasgow  | G11 9QX  | Flat  | 3     | 350  | CO40    |         | B003     |
| PG36       | 2 Manor Rd    | Glasgow  | G32 4QX  | Flat  | 3     | 375  | CO93    | SG37    | B003     |
| PG21       | 18 Dale Rd    | Glasgow  | G12      | House | 5     | 600  | CO87    | SG37    | B003     |
| PG16       | 5 Novar Dr    | Glasgow  | G12 9AX  | Flat  | 4     | 450  | CO93    | SG14    | B003     |

# SQL Stored Procedures

- ▶ A stored procedure is a prepared SQL code that you can save.
- ▶ The code can be reused repeatedly.
- ▶ So if you have an SQL query that you write again and again, save it as a stored procedure, and then just call it to execute it.
- ▶ You can also pass parameters to a stored procedure, so that the stored procedure can act based on the parameter value(s) that is passed.
- ▶ A stored procedure can consist of more than one query.

## ► Syntax

- CREATE PROCEDURE *procedure\_name*  
AS  
*sql\_statement (s)*  
;

## ► Execution

- EXEC *procedure\_name*;

# Simple Example

- ▶ CREATE PROCEDURE SelectAllStaff  
AS  
SELECT \* FROM Staff;
- ▶ exec selectallStaff;

|   | staffNo | fName | lName | position   | sex | DOB        | salary | branchNo |
|---|---------|-------|-------|------------|-----|------------|--------|----------|
| 1 | SA9     | Mary  | Howe  | Assistant  | F   | 1970-02-19 | 9000   | B007     |
| 2 | SG14    | David | Ford  | Supervisor | M   | 1958-11-24 | 18000  | B003     |
| 3 | SG37    | Ann   | Beech | Assistant  | F   | 1960-10-11 | 12000  | B003     |
| 4 | SG5     | Susan | Brand | Manager    | F   | 1940-06-03 | 24000  | B003     |
| 5 | SL21    | John  | White | Manager    | M   | 1945-10-01 | 30000  | B005     |
| 6 | SL41    | Julie | Lee   | Assistant  | F   | 1965-06-13 | 9000   | B005     |

# Stored Procedure With One Parameter

► CREATE PROCEDURE SelectPositionWiseStaff @position varchar(30)

AS

SELECT \* FROM staff WHERE position = @position;

► exec selectpositionwisestaff 'manager'

|   | staffNo | fName | lName | position | sex | DOB        | salary | branchNo |
|---|---------|-------|-------|----------|-----|------------|--------|----------|
| 1 | SG5     | Susan | Brand | Manager  | F   | 1940-06-03 | 24000  | B003     |
| 2 | SL21    | John  | White | Manager  | M   | 1945-10-01 | 30000  | B005     |

# Stored Procedure With Multiple Parameters

- ▶ `CREATE PROCEDURE PropertiesBranchstaff @staffno varchar(30), @branchno varchar(10)`  
`AS`  
`SELECT propertyno FROM PropertyForRent WHERE staffno = @staffno AND branchno = @branchno;`
- ▶ `exec PropertiesBranchstaff @staffno = 'SG37', @branchno = 'B003';`

|   | propertyno |
|---|------------|
| 1 | PG21       |
| 2 | PG36       |

# Stored Procedure With Multiple statements

- ▶ CREATE PROCEDURE SelectPositionWiseStaff\_branch @position varchar(30)  
AS  
SELECT \* FROM staff WHERE position = @position;  
SELECT \* FROM branch where branchno in (select branchno from staff where position= @position);
- ▶ execute SelectPositionWiseStaff\_branch 'manager';

|   | staffNo | fName | lName | position | sex | DOB        | salary | branchNo |
|---|---------|-------|-------|----------|-----|------------|--------|----------|
| 1 | SG5     | Susan | Brand | Manager  | F   | 1940-06-03 | 24000  | B003     |
| 2 | SL21    | John  | White | Manager  | M   | 1945-10-01 | 30000  | B005     |

|   | branchNo | street      | city    | postCode |
|---|----------|-------------|---------|----------|
| 1 | B003     | 163 Main St | Glasgow | G11 9QX  |
| 2 | B005     | 22 Deer Rd  | London  | SW1 4EH  |

# Salary updation of an Assistant SL41

- ▶ CREATE PROCEDURE salaryUpdationStaff @position varchar(30), @staffno varchar(30), @salary int  
AS  
SELECT \* FROM staff WHERE position = @position AND staffno=@staffno;  
update staff set salary = salary+(salary \*@salary/100) where position = @position AND  
staffno=@staffno;  
SELECT \* FROM staff WHERE position = @position AND staffno=@staffno;
- ▶ execute salaryUpdationStaff 'Assistant', 'SL41', 5;

|   | staffNo | fName | lName | position  | sex | DOB        | salary | branchNo |
|---|---------|-------|-------|-----------|-----|------------|--------|----------|
| 1 | SL41    | Julie | Lee   | Assistant | F   | 1965-06-13 | 12000  | B005     |

|   | staffNo | fName | lName | position  | sex | DOB        | salary | branchNo |
|---|---------|-------|-------|-----------|-----|------------|--------|----------|
| 1 | SL41    | Julie | Lee   | Assistant | F   | 1965-06-13 | 12600  | B005     |

# CSC270- Database Systems

## Lab 7(part 1)

### Spring 2026

Mahrukh Saleem

[mahrukhsaleem@cuilahore.edu.pk](mailto:mahrukhsaleem@cuilahore.edu.pk)

**Branch**

| branchNo | street       | city     | postcode |
|----------|--------------|----------|----------|
| B005     | 22 Deer Rd   | London   | SW1 4EH  |
| B007     | 16 Argyll St | Aberdeen | AB2 3SU  |
| B003     | 163 Main St  | Glasgow  | G11 9QX  |
| B004     | 32 Manse Rd  | Bristol  | BS99 1NZ |
| B002     | 56 Clover Dr | London   | NW10 6EU |

**Staff**

| staffNo | fName | lName | position   | sex | DOB       | salary | branchNo |
|---------|-------|-------|------------|-----|-----------|--------|----------|
| SL21    | John  | White | Manager    | M   | 1-Oct-45  | 30000  | B005     |
| SG37    | Ann   | Beech | Assistant  | F   | 10-Nov-60 | 12000  | B003     |
| SG14    | David | Ford  | Supervisor | M   | 24-Mar-58 | 18000  | B003     |
| SA9     | Mary  | Howe  | Assistant  | F   | 19-Feb-70 | 9000   | B007     |
| SG5     | Susan | Brand | Manager    | F   | 3-Jun-40  | 24000  | B003     |
| SL41    | Julie | Lee   | Assistant  | F   | 13-Jun-65 | 9000   | B005     |

**PrivateOwner**

| ownerNo | fName | lName  | address                       | telNo         | eMail             |
|---------|-------|--------|-------------------------------|---------------|-------------------|
| CO46    | Joe   | Keogh  | 2 Fergus Dr, Aberdeen AB2 7SX | 01224-861212  | jkeogh@lhh.com    |
| CO87    | Carol | Farrel | 6 Achray St, Glasgow G32 9DX  | 0141-357-7419 | cfarrel@gmail.com |
| CO40    | Tina  | Murphy | 63 Well St, Glasgow G42       | 0141-943-1728 | tinam@hotmail.com |
| CO93    | Tony  | Shaw   | 12 Park Pl, Glasgow G4 0QR    | 0141-225-7025 | tony.shaw@ark.com |

**PropertyForRent**

| propertyNo | street        | city     | postcode | type  | rooms | rent | ownerNo | staffNo | branchNo |
|------------|---------------|----------|----------|-------|-------|------|---------|---------|----------|
| PA14       | 16 Holhead    | Aberdeen | AB7 5SU  | House | 6     | 650  | CO46    | SA9     | B007     |
| PL94       | 6 Argyll St   | London   | NW2      | Flat  | 4     | 400  | CO87    | SL41    | B005     |
| PG4        | 6 Lawrence St | Glasgow  | G11 9QX  | Flat  | 3     | 350  | CO40    |         | B003     |
| PG36       | 2 Manor Rd    | Glasgow  | G32 4QX  | Flat  | 3     | 375  | CO93    | SG37    | B003     |
| PG21       | 18 Dale Rd    | Glasgow  | G12      | House | 5     | 600  | CO87    | SG37    | B003     |
| PG16       | 5 Novar Dr    | Glasgow  | G12 9AX  | Flat  | 4     | 450  | CO93    | SG14    | B003     |

# ASCII Character

- ▶ SELECT fname, ASCII(fname) AS NumCodeOfFirstChar
- ▶ FROM staff;

|   | fname | NumCodeOfFirstChar |
|---|-------|--------------------|
| 1 | Mary  | 77                 |
| 2 | David | 68                 |
| 3 | Ann   | 65                 |
| 4 | Susan | 83                 |
| 5 | John  | 74                 |
| 6 | Julie | 74                 |

# SQL Server CHAR() Function

- ▶ `SELECT CHAR(65) AS CodeToCharacter;`
- ▶ Non-unicode characters 1 byte

|   | CodeToCharacter |
|---|-----------------|
| 1 | A               |

# SQL Server CHARINDEX() Function

- ▶ The CHARINDEX() function searches for a substring in a string and returns the position.
- ▶ If the substring is not found, this function returns 0.
- ▶ **Note:** This function performs a case-insensitive search.
- ▶ CHARINDEX(*substring*, *string*, *start*)
- ▶ SELECT CHARINDEX('mer', 'Customer') AS MatchPosition;

|   | MatchPosition |
|---|---------------|
| 1 | 6             |

# SQL Server CONCAT() Function

- ▶ `CONCAT(string1, string2, ..., string_n)`
- ▶ `SELECT CONCAT('SQL', 'is', 'fun!');`
- ▶ `SELECT CONCAT('SQL', ' ', 'is', ' ', 'fun!');`



| (No column name) |             |
|------------------|-------------|
| 1                | SQL is fun! |



# SQL Server DATALENGTH() Function

- ▶ The DATALENGTH() function returns the number of bytes used to represent an expression.
- ▶ **Note:** The DATALENGTH() function counts both leading and trailing spaces when calculating the length of the expression.
- ▶ DATALENGTH(*expression*)
- ▶ SELECT DATALENGTH(' SQL is a fun ');

|   | (No column name) |
|---|------------------|
| 1 | 22               |

# SQL Server LEFT() Function

- ▶ The LEFT() function extracts a number of characters from a string (starting from left).
- ▶ LEFT(*string*, *number\_of\_chars*)
- ▶ SELECT LEFT(lname, 3) AS ExtractString FROM staff;

|   | ExtractString |
|---|---------------|
| 1 | How           |
| 2 | For           |
| 3 | Bee           |
| 4 | Bra           |
| 5 | Whi           |
| 6 | Lee           |

# SQL Server LEN() Function

- ▶ Count leading spaces but not trailing
- ▶ `SELECT LEN(' W3Schools.com ');`

|   | (No column name) |
|---|------------------|
| 1 | 16               |

# SQL Server LOWER() / UPPER() Function

- ▶ `SELECT LOWER('SQL Tutorial is FUN!');`
- ▶ `SELECT UPPER('sql tutorial is a fun');`

|   | (No column name)     |
|---|----------------------|
| 1 | sql tutorial is fun! |

# SQL Server LTRIM()/RTRIM() Function

- ▶ The LTRIM() function removes leading spaces from a string.
- ▶ The RTRIM() function remove trailing spaces from a string:
- ▶ LTRIM(*string*)
- ▶ SELECT LTRIM(' SQL Tutorial') AS LeftTrimmedString;

|   | Left TrimmedString |
|---|--------------------|
| 1 | SQL Tutorial       |

# SQL Server NCHAR() Function

- ▶ The NCHAR() function returns the Unicode character based on the number code.
- ▶ NCHAR(*number\_code*)
- ▶ SELECT NCHAR(65) AS NumberCodeToUnicode;

|   | NumberCodeToUnicode |
|---|---------------------|
| 1 | A                   |

# SQL Server REVERSE() Function

- ▶ The REVERSE() function reverses a string and returns the result.
- ▶ REVERSE(*string*)
- ▶ SELECT REVERSE(fname) FROM staff

|   | (No column name) |
|---|------------------|
| 1 | yraM             |
| 2 | divaD            |
| 3 | nnA              |
| 4 | nasuS            |
| 5 | nhoJ             |
| 6 | eiluJ            |

# SQL Server SPACE() Function

- ▶ The SPACE() function returns a string of the specified number of space characters.
- ▶ SPACE(*number*)

# SQL Server STR() Function

- ▶ The STR() function returns a number as a string.
- ▶ *STR(number, length, decimals)*
- ▶ Number: Required. The number to convert to a string
- ▶ Length: Optional. The length of the returning string. Default value is 10
- ▶ Decimal: Optional. The number of decimals to display in the returning string. Default value is 0
- ▶ `SELECT STR(185.476, 6, 2);`

| (No column name) |        |
|------------------|--------|
| 1                | 185.48 |

# SQL Server SUBSTRING() Function

- ▶ The SUBSTRING() function extracts some characters from a string.
- ▶ SUBSTRING(*string*, *start*, *length*)
- ▶ String: Required. The string to extract from
- ▶ Start: Required. The start position. The first position in *string* is 1
- ▶ Length: Required. The number of characters to extract. Must be a positive number
- ▶ SELECT SUBSTRING(address, 1, 10) AS ExtractString FROM privateowner;

|   | ExtractString |
|---|---------------|
| 1 | 63 Well St    |
| 2 | 2 Fergus D    |
| 3 | 6 Achray S    |
| 4 | 12 Park Pl    |

# CSC270 - Database Systems

## Lab 7(part 2)

### Spring 2026

Mahrukh Saleem

[mahrukhsaleem@cuilahore.edu.pk](mailto:mahrukhsaleem@cuilahore.edu.pk)

# SQL Server ABS() Function

- ▶ The ABS() function returns the absolute value of a number.
- ▶ `ABS(number)`
- ▶ `SELECT Abs(-243.5) AS AbsNum;`

| AbsNum |
|--------|
| 243.5  |

# SQL Server CEILING()/FLOOR Function

- ▶ The CEILING() function returns the smallest integer value that is larger than or equal to a number.
- ▶ **Tip:** Also look at the [FLOOR\(\)](#) and [ROUND\(\)](#) functions.
- ▶ CEILING(*number*)
- ▶ SELECT CEILING(25.75) AS CeilValue;
- ▶ SELECT FLOOR(25.75) AS FloorValue;

| CeilValue |
|-----------|
| 26        |

# SQL Server ROUND() Function

- ▶ The ROUND() function rounds a number to a specified number of decimal places.
- ▶ ROUND(*number*, *decimals*, *operation*)

| Parameter        | Description                                                                                                                                                                   |
|------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>number</i>    | Required. The number to be rounded                                                                                                                                            |
| <i>decimals</i>  | Required. The number of decimal places to round <i>number</i> to                                                                                                              |
| <i>operation</i> | Optional. If 0, it rounds the result to the number of <i>decimal</i> . If another value than 0, it truncates the result to the number of <i>decimals</i> . Default value is 0 |

- ▶ SELECT ROUND(235.415, 2, 1) AS RoundValue;
- ▶ SELECT ROUND(235.415, 2, 0) AS RoundValue;
- ▶ SELECT ROUND(235.415, -1) AS RoundValue;
- ▶ <https://www.youtube.com/watch?v=Mx6GcNVECH0>

**RoundValue**

235.410

**RoundValue**

235.420

**RoundValue**

240.000

# SQL Server POWER() Function

- ▶ The POWER() function returns the value of a number raised to the power of another number.
- ▶ POWER(*a*, *b*)

| Parameter | Description                       |
|-----------|-----------------------------------|
| <i>a</i>  | Required. A number (the base)     |
| <i>b</i>  | Required. A number (the exponent) |

- ▶ SELECT POWER(8, 3) as Value;

**Value**

**512**

# SQL Server SIGN() Function

- ▶ The SIGN() function returns the sign of a number.
- ▶ This function will return one of the following:
- ▶ If number > 0, it returns 1
- ▶ If number = 0, it returns 0
- ▶ If number < 0, it returns -1
- ▶ `SIGN(number)`
- ▶ `SELECT SIGN(-12) AS TypeOfValue;`

**TypeOfValue**

**-1**

# SQL Server SQUARE() Function

- ▶ The SQUARE() function returns the square of a number.
- ▶ `SQUARE(number)`
- ▶ `SELECT SQUARE(15);`

| SquareOfValue |
|---------------|
| 225.0         |

# SQL Server SQRT() Function

- ▶ The SQRT() function returns the square root of a number.
- ▶ `SQRT(number)`
- ▶ `SELECT SQRT(13)SquareRoot;`

---

**SquareRoot**

3.6055512754639891

# SQL Server SIN() Function

- ▶ The SIN() function returns the sine of a number.
- ▶ `SIN(number)`
- ▶ `SELECT SIN(2) AS SineValue;`

**SineValue**

0.90929742682568171

- ▶ The COS() function returns the cosine of a number.
- ▶ The TAN() function returns the tangent of a number.
- ▶ The COT() function returns the cotangent of a number.
- ▶ Etc.

# SQL Server RAND() Function

- ▶ The RAND() function returns a random number between 0 (inclusive) and 1 (exclusive).

- ▶ RAND(*seed*);

| Parameter   | Description                                                                                                                                        |
|-------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>seed</i> | Optional. If seed is specified, it returns a repeatable sequence of random numbers. If no seed is specified, it returns a completely random number |

- ▶ SELECT RAND() AS RandomNumber;

**RandomNumber**

0.74856666001288341

- ▶ <https://www.youtube.com/watch?v=Gvm2Ao6YmrU>

- ▶ SELECT RAND(6) AS RandomNumber;

**RandomNumber**

0.71368515806921451

# SQL Server LOG() Function

- ▶ The LOG() function returns the natural logarithm of a specified *number*, or the logarithm of the *number* to the specified *base*.
- ▶ LOG(*number*, *base*) -- Syntax for SQL Server
- ▶ SELECT LOG(2, 4) AS Log;

Log

0.5

# SQL Server LOG10() Function

- ▶ The LOG10() function returns the natural logarithm of a number to base 10.
- ▶ LOG10(*number*)
- ▶ SELECT LOG10(4.5) AS Log10;

---

**Log10**

0.65321251377534373

# Other Functions

- ▶ Sum()
- ▶ Count
- ▶ Average()
- ▶ Min()
- ▶ Max()

# CSC270- Database Systems

## Lab 8

### Spring 2026

Mahrukh Saleem

[mahrukhsaleem@cuilahore.edu.pk](mailto:mahrukhsaleem@cuilahore.edu.pk)

# Triggers

- ▶ SQL triggers are stored procedures that automatically execute in response to certain events in a specific table or view in a database. They are used to maintain the integrity of the data, enforce business rules, and automate tasks. We can set triggers to fire after or instead of an INSERT, UPDATE, or DELETE operation.

# Types of triggers

- ▶ **DML Triggers (Data Manipulation Language):** These triggers fire in response to INSERT, UPDATE, or DELETE operations on a table or view.
- ▶ **AFTER Triggers:**
  - ▶ Also known as **FOR triggers**.
  - ▶ Execute **after** the triggering action (e.g., after an INSERT, UPDATE, or DELETE).
  - ▶ Cannot be defined on views.
- ▶ **INSTEAD OF Triggers:**
  - ▶ Execute **instead of** the triggering action.
  - ▶ Can be defined on both tables and views.

# DDL Triggers

- ▶ **DDL Triggers (Data Definition Language Triggers):** These triggers fire in response to DDL statements such as CREATE, ALTER, or DROP operations (on databases or tables).
- ▶ **Purpose:** To audit or prevent schema changes.
- ▶ **Example:**
  - ▶ Log every time a table is dropped.
  - ▶ Restrict users from altering database objects.

# Example

## ► Creating a basic trigger

```
CREATE TRIGGER Trigger_Name
ON Table_Name
AFTER INSERT, UPDATE, DELETE
AS
BEGIN
-- SQL statements to execute after the event
PRINT 'Trigger executed'
END
```

## ► Example

```
CREATE TRIGGER trg_AfterInsert
ON Employees
AFTER INSERT
AS
BEGIN
PRINT 'A new record has been inserted into Employees table'
END
```

# INSTEAD OF Trigger

```
CREATE TRIGGER Trigger_Name
ON Table_Name
INSTEAD OF INSERT, UPDATE, DELETE
AS
BEGIN
-- SQL statements to execute instead of the event
PRINT 'Instead of trigger executed'
END
```

## ► Example

```
CREATE TRIGGER trg_InsteadOfDelete
ON Employees
INSTEAD OF DELETE
AS
BEGIN
PRINT 'Delete operation was prevented'
END
```

# DDL Triggers

```
CREATE TRIGGER trg_DDLTrigger
ON DATABASE
FOR CREATE_TABLE, DROP_TABLE
AS
BEGIN
PRINT 'DDL operation detected'
END
```

## ► Example

```
CREATE TRIGGER trg_PreventDropTable
ON DATABASE
FOR DROP_TABLE
AS BEGIN
PRINT 'Dropping tables is not allowed!'
ROLLBACK;
END
```

# Trigger Special Tables

- ▶ Triggers in SQL Server use two special tables:
- ▶ **INSERTED**: Contains the new data for INSERT or UPDATE operations.
- ▶ **DELETED**: Contains the old data for DELETE or UPDATE operations.
- ▶ These tables allow you to compare old and new values or retrieve affected rows.
- ▶ Example

```
CREATE TRIGGER trg_AuditUpdate
ON Employees
AFTER UPDATE
AS
BEGIN
SELECT * FROM INSERTED; -- New values
SELECT * FROM DELETED; -- Old values
END
```